

14.9 Consider the idea of implementing modular hashing for integer keys with the code $(a * x) \% M$, where a is an arbitrary fixed prime. Does this change mix up the bits sufficiently well that you can use nonprime M ?

14.10 Prove that $((ax) \bmod M) + b \bmod M = (ax + b) \bmod M$, assuming that a , b , x , and M are all nonnegative integers.

▷ **14.11** If you use the words from a text file, such as a book, in Exercise 14.7, you are unlikely to get a good χ^2 statistic. Explain why this assertion is true.

14.12 Use your program from Exercise 14.5 to evaluate the hash function $97 * x \% M$, for all table sizes between 100 and 200, using 10^3 random positive integers less than 10^6 as keys.

14.13 Use your program from Exercise 14.5 to evaluate the hash function $97 * x \% M$, for all table sizes between 100 and 200, using the integers between 10^2 and 10^3 as keys.

14.14 Use your program from Exercise 14.5 to evaluate the hash function $100 * x \% M$, for all table sizes between 100 and 200, using 10^3 random positive integers less than 10^6 as keys.

14.15 Do Exercises 14.12 and 14.14, but use the simpler criterion of rejecting hash functions that produce any value more than $3N/M$ times.

14.2 Separate Chaining

The hash functions discussed in Section 14.1 convert keys into table addresses; the second component of a hashing algorithm is to decide how to handle the case when two keys hash to the same address. The most straightforward method is to build, for each table address, a linked list of the items whose keys hash to that address. This approach leads directly to the generalization of elementary list search (see Chapter 12) that is given in Program 14.3. Rather than maintaining a single list, we maintain M lists.

This method is traditionally called *separate chaining*, because items that collide are chained together in separate linked lists. An example is depicted in Figure 14.6. As with elementary sequential search, we can choose to keep the lists in sorted order, or we can leave them unordered. The same basic tradeoffs as those discussed in Section 12.3 apply, but, for separate chaining, the time savings are less significant (because the lists are short) and the space usage is more significant (because there are so many lists).

We might be using a header node to streamline the code for insertion into an ordered list, but we might not want to use M header

A S E R C H I N G X M P L
0 2 0 4 4 4 2 2 1 2 4 3 3

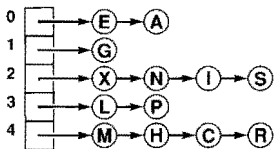


Figure 14.6
Hashing with separate chaining

This diagram shows the result of inserting the keys A S E R C H I N G X M P L into an initially empty hash table with separate chaining (unordered lists), using the hash values given at the top. The A goes into list 0, then the S goes into list 2, then the E goes into list 0 (at the front, to keep the insertion time constant), then the R goes into list 4, and so forth.

Program 14.3 Hashing with separate chaining

This symbol-table implementation is based on replacing the STinit, STsearch, STinsert, and STdelete functions in the linked-list-based symbol table of Program 12.5 with the functions given here, and replacing the link head with an array of links heads. We use the same recursive list search and deletion procedures as in Program 12.5, but we maintain M lists, with head links in heads, using a hash function to choose among the lists. The STinit function sets M such that we expect the lists to have about five items each; therefore the other operations require just a few probes.

```

static link *heads, z;
static int N, M;
void STinit(int max)
{ int i;
  N = 0; M = max/5;
  heads = malloc(M*sizeof(link));
  z = NEW(NULLitem, NULL);
  for (i = 0; i < M; i++) heads[i] = z;
}
Item STsearch(Key v)
{ return searchR(heads[hash(v, M)], v); }
void STinsert(Item item)
{ int i = hash(key(item), M);
  heads[i] = NEW(item, heads[i]); N++; }
void STdelete(Item item)
{ int i = hash(key(item), M);
  heads[i] = deleteR(heads[i], item); }

```

nodes for individual lists in separate chaining. Indeed, we could even eliminate the M links to the lists by having the first nodes in the lists comprise the table (see Exercise 14.20).

For a search miss, we can assume that the hash function scrambles the key values sufficiently well that each of the M lists is equally likely to be searched. Then the performance characteristics that we studied in Section 12.3 apply, for each list.

Property 14.1 *Separate chaining reduces the number of comparisons for sequential search by a factor of M (on the average), using extra space for M links.*

The average length of the lists is N/M . As described in Chapter 12, successful searches are expected to go about halfway down some list. Unsuccessful searches go to the end of a list if the lists are unordered, halfway down a list if the lists are kept in order. ■

Most often, we use unordered lists for separate chaining, because that approach is both easy to implement and efficient: *insert* takes constant time and *search* takes time proportional to N/M . If huge numbers of search misses are expected, we can speed up the misses by a factor of 2 by keeping the lists ordered, at the cost of a slower *insert*.

As stated, Property 14.1 is a trivial result, because the average length of the lists is N/M , no matter how the items are distributed among the lists. For example, suppose that all the items fall onto the first list. Then, the average length of the lists is $(N+0+0+\dots+0)/M = N/M$. The real reason that hashing is useful in practice is that *each* list is extremely likely to have about N/M items.

Property 14.2 *In a separate-chaining hash table with M lists and N keys, the probability that the number of keys in each list is within a small constant factor of N/M is extremely close to 1.*

We briefly consider this classical analysis, for readers who are familiar with basic probabilistic analysis. The probability that a given list will have k items on it is

$$\binom{N}{k} \left(\frac{1}{M}\right)^k \left(1 - \frac{1}{M}\right)^{N-k}$$

by an elementary argument. We choose k out of the N items: Those k items hash to the given list with probability $1/M$, and the other $N - k$ items do not hash to the given list with probability $1 - (1/M)$. In terms of $\alpha = N/M$, we can rewrite this expression as

$$\binom{N}{k} \left(\frac{\alpha}{N}\right)^k \left(1 - \frac{\alpha}{N}\right)^{N-k},$$

which, by the classical Poisson approximation, is less than

$$\frac{\alpha^k e^{-\alpha}}{k!}.$$

From this result, it follows that the probability that a list has more than $t\alpha$ items on it is less than

$$\left(\frac{\alpha e}{t}\right)^t e^{-\alpha}.$$

This probability is extremely small for practical ranges of the parameters. For example, if the average length of the lists is 20, the probability that we will hash to some list with more than 40 items on it is less than $(20e/2)^2 e^{-20} \approx 0.0000016$. ■

The foregoing analysis is an example of a classical *occupancy problem*, where we consider N balls thrown randomly into one of M urns, and analyze how the balls are distributed among the urns. Classical mathematical analysis of these problems tells us many other interesting facts that are relevant to the study of hashing algorithms. For example, the Poisson approximation tells us that the number of empty lists is about $e^{-\alpha}$. A more interesting result tells us that the average number of items inserted before the first collision occurs is about $\sqrt{\pi M/2} \approx 1.25\sqrt{M}$. This result is the solution to the classical *birthday problem*. For example, the same analysis tells us, for $M = 365$, that the average number of people we need to check before finding two with the same birthday is about 24. A second classical result tells us that the average number of items inserted before each list has at least one item is about MH_M . This result is the solution to the classical *coupon collector problem*. For example, the same analysis tells us, for $M = 1280$, that we would expect to collect 9898 baseball cards (coupons) before getting one for each of 40 players on each of 32 teams in a series.

These research results are indicative of the properties of hashing that have been analyzed. In practice, they tell us that we can use separate chaining with great confidence, if the hash function produces values that approximate random ones (*see reference section*).

In a separate-chaining implementation, we typically choose M to be small enough that we are not wasting a huge area of contiguous memory with empty links, but large enough that sequential search is the most efficient method for the lists. Hybrid methods (such as using binary trees instead of linked lists) are probably not worth the trouble. As a rule of thumb, we might choose M to be about one-fifth or one-tenth the number of keys expected to be in the table, so that the lists are expected to contain about five or 10 keys each. One of the virtues

of separate chaining is that this decision is not critical: if more keys arrive than expected, then searches will take a little longer than if we had chosen a bigger table size ahead of time; if fewer keys are in the table, then we have extra-fast *search* with perhaps a small amount of wasted space. When space is not a critical resource, M can be chosen sufficiently large that search time is constant; when space *is* a critical resource, we still can get a factor of M improvement in performance by choosing M to be as large as we can afford.

The comments in the previous paragraph apply to search time. In practice, unordered lists are normally used for separate chaining, for two primary reasons. First, as we have mentioned, *insert* is extremely fast: We compute the hash function, allocate memory for the node, and link in the node at the beginning of the appropriate list. In many applications, the memory-allocation step is not needed (because the items inserted into the symbol table may be existing records with available link fields), and we are left with perhaps three or four machine instructions for *insert*. The second important advantage of using the unordered-list implementation in Program 14.3 is that the lists all function as stacks, so we can easily remove the most recently inserted items, which are at the front of the lists (see Exercise 14.21). This operation is an important one when we are implementing a symbol table with nested scopes, for example in a compiler.

As in several previous implementations, we implicitly give the client a choice for handling duplicate keys. A client like Program 12.10 might *search* to check for duplicates before any *insert*, thus ensuring that the table does not contain any duplicate keys. Another client might avoid the cost of this *search* by leaving duplicates in the table, thus achieving fast *insert* operations.

Generally, hashing is not appropriate for use in applications where implementations for the *sort* and *select* ADT operations are required. However, hashing is often used for the typical situation where we need to use a symbol table with potentially a large number of *search*, *insert*, and *delete* operations, then to print out the items in order of their keys once, at the end. One example of such an application is a symbol table in a compiler; another is a program to remove duplicates, such as Program 12.10. To handle this situation in an unordered-list implementation of separate chaining, we would have to use one of the sorting methods described in Chapters 6 through 10; in

an ordered-list implementation, we could accomplish the sort in time proportional to $N \lg M$ with list mergesort (see Exercise 14.23).

Exercises

- ▷ **14.16** How long could it take in the worst case to insert N keys into an initially empty table, using separate chaining with (i) unordered lists and (ii) ordered lists?
- ▷ **14.17** Give the contents of the hash table that results when you insert items with the keys EASYQUESTION in that order into an initially empty table of $M = 5$ lists, using separate chaining with unordered lists. Use the hash function $11k \bmod M$ to transform the k th letter of the alphabet into a table index.
- ▷ **14.18** Answer Exercise 14.17, but use ordered lists. Does your answer depend on the order in which you insert the items?
- **14.19** Write a program that inserts N random integers into a table of size $N/100$ using separate chaining, then finds the length of the shortest and longest lists, for $N = 10^3, 10^4, 10^5$, and 10^6 .
- 14.20** Modify Program 14.3 to eliminate the head links by representing the symbol table as an array of STnodes (each table entry is the first node in its list).
- 14.21** Modify Program 14.3 to include an integer field for each item that is set to the number of items in the table at the time the item is inserted. Then implement a function that deletes all items for which the field is greater than a given integer N .
- 14.22** Modify the implementation of STsearch in Program 14.3 to visit all the items with keys equal to a given key, in the same manner as STsort.
- 14.23** Implement a symbol table using separate chaining with ordered lists (with a fixed table of size 97) that supports the *initialize*, *count*, *search*, *insert*, *delete*, *join*, *select*, and *sort* operations for a first-class symbol-table ADT, with support for client handles (see Exercises 12.4 and 12.5).

14.3 Linear Probing

If we can estimate in advance the number of elements to be put into the hash table and have enough contiguous memory available to hold all the keys with some room to spare, then it is probably not worthwhile to use any links at all in the hash table. Several methods have been devised that store N items in a table of size $M > N$, relying on empty places in the table to help with collision resolution. Such methods are called *open-addressing* hashing methods.

The simplest open-addressing method is called *linear probing*: when there is a collision (when we hash to a place in the table that is already occupied with an item whose key is not the same as the search key), then we just check the next position in the table. It is customary to refer to such a check (determining whether or not a given table position holds an item with key equal to the search key) as a *probe*. Linear probing is characterized by identifying three possible outcomes of a probe: if the table position contains an item whose key matches the search key, then we have a search hit; if the table position is empty, then we have a search miss; otherwise (if the table position contains an item whose key does not match the search key) we just probe the table position with the next higher index, continuing (wrapping back to the beginning of the table if we reach the end) until either the search key or an empty table position is found. If an item containing the search key is to be inserted following an unsuccessful search, then we put it into the empty table space that terminated the search. Program 14.4 is an implementation of the symbol-table ADT using this method. The process of constructing a hash table for a sample set of keys using linear probing is shown in Figure 14.7.

As with separate chaining, the performance of open-addressing methods is dependent on the ratio $\alpha = N/M$, but we interpret it differently. For separate chaining, α is the average number of items per list and is generally larger than 1. For open addressing, α is the fraction of those table positions that are occupied; it must be less than 1. We sometimes refer to α as the *load factor* of the hash table.

For a sparse table (small α), we expect most searches to find an empty position with just a few probes. For a nearly full table (α close to 1), a search could require a huge number of probes, and could even fall into an infinite loop when the table is completely full. Typically, we insist that the table *not* be allowed to become nearly full when using linear probing, to avoid long search times. That is, rather than using extra memory for links, we use it for extra space in the hash table that shortens probe sequences. The table size for linear probing is greater than for separate chaining, since we must have $M > N$, but the total amount of memory space used may be less, since no links are used. We will discuss space-usage comparisons in detail in Section 14.5; for the moment, we consider the analysis of the running time of linear probing as a function of α .

Program 14.4 Linear probing

This symbol-table implementation keeps items in a table twice the size of the maximum number of items expected, initialized to `NULLitem`. The table holds the items themselves; if the items are large, we can modify the item type to hold links to the items.

To insert a new item, we hash to a table position and scan to the right to find an unoccupied position, using the macro `null` to check whether a table position is unoccupied. To search for an item with a given key, we go to the key hash position and scan to look for a match, stopping when we hit an unoccupied position.

The `STinit` function sets `M` such that we may expect the table to be less than half full, so the other operations will require just a few probes, if the hash function produces values that are sufficiently close to random ones.

```
#include <stdlib.h>
#include "Item.h"
#define null(A) (key(st[A]) == key(NULLitem))
static int N, M;
static Item *st;
void STinit(int max)
{ int i;
  N = 0; M = 2*max;
  st = malloc(M*sizeof(Item));
  for (i = 0; i < M; i++) st[i] = NULLitem;
}
int STcount() { return N; }
void STinsert(Item item)
{ Key v = key(item);
  int i = hash(v, M);
  while (!null(i)) i = (i+1) % M;
  st[i] = item; N++;
}
Item STsearch(Key v)
{ int i = hash(v, M);
  while (!null(i))
    if (eq(v, key(st[i])) return st[i];
    else i = (i+1) % M;
  return NULLitem;
}
```


The average cost of linear probing depends on the way in which the items cluster together into contiguous groups of occupied table cells, called *clusters*, when they are inserted. Consider the following two extremes in a linear probing table that is half full ($M = 2N$): In the best case, table positions with even indices could be empty, and table positions with odd indices could be occupied. In the worst case, the first half of the table positions could be empty, and the second half occupied. The average length of the clusters in both cases is $N/(2N) = 1/2$, but the average number of probes for an unsuccessful search is 1 (all searches take at least 1 probe) plus

$$(0 + 1 + 0 + 1 + \dots)/(2N) = 1/2$$

in the best case, and is 1 plus

$$(N + (N - 1) + (N - 2) + \dots)/(2N) \approx N/4$$

in the worst case.

Generalizing this argument, we find that the average number of probes for an unsuccessful search is proportional to the *squares* of the lengths of the clusters. We compute the average by computing the cost of a search miss starting at each position in the table, then dividing the total by M . All search misses take at least 1 probe, so we count the number of probes after the first. If a cluster is of length t , then the expression

$$(t + (t - 1) + \dots + 2 + 1)/M = t(t + 1)/(2M)$$

counts the contribution of that cluster to the grand total. The sum of the cluster lengths is N , so, adding this cost for all cells in the table, we find that the total average cost for a search miss is $1 + N/(2M)$ plus the sum of the squares of the lengths of the clusters, divided by $2M$. Given a table, we can quickly compute the average cost of unsuccessful search in that table (see Exercise 14.28), but the clusters are formed by a complicated dynamic process (the linear-probing algorithm) that is difficult to characterize analytically.

Property 14.3 *When collisions are resolved with linear probing, the average number of probes required to search in a hash table of size M that contains $N = \alpha M$ keys is about*

$$\frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right) \quad \text{and} \quad \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$$

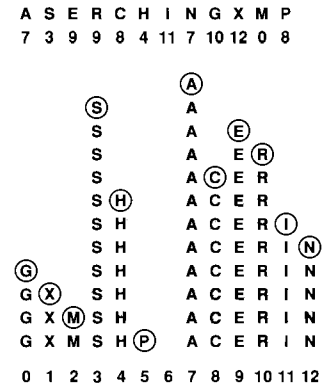


Figure 14.7
Hashing with linear probing

This diagram shows the process of inserting the keys A S E R C H I N G X M P into an initially empty hash table of size 13 with open addressing, using the hash values given at the top and resolving collisions with linear probing. First, the A goes into position 7, then the S goes into position 3, then the E goes into position 9, then the R goes into position 10 after a collision at position 9, and so forth. Probe sequences that run off the right end of the table continue on the left end: for example, the final key inserted, the P, hashes to position 8, then ends up in position 5 after collisions at positions 8 through 12, then 0 through 5. All table positions not probed are shaded.

for hits and misses, respectively.

Despite the relatively simple form of the results, precise analysis of linear probing is a challenging task. Knuth’s completion of it in 1962 was a landmark in the analysis of algorithms (see reference section). ■

These estimates lose accuracy as α approaches 1, but we do not need them for that case, because we should not be using linear probing in a nearly full table in any event. For smaller α , the equations are sufficiently accurate. The following table summarizes the expected number of probes for search hits and misses with linear probing:

load factor (α)	1/2	2/3	3/4	9/10
search hit	1.5	2.0	3.0	5.5
search miss	2.5	5.0	8.5	55.5

Search misses are always more expensive than hits, and both require only a few probes, on the average, in a table that is less than half full.

As we did with separate chaining, we leave to the client the choice of whether or not to keep items with duplicate keys in the table. Such items do not necessarily appear in contiguous positions in a linear probing table—other items with the same hash value can appear among items with duplicate keys.

By the very nature of the way the table is constructed, the keys in a table built with linear probing are in random order. The *sort* and *select* ADT operations require starting from scratch with one of the methods described in Chapters 6 through 10, so linear probing is not appropriate for applications where these operations are performed frequently.

How do we delete a key from a table built with linear probing? We cannot just remove it, because items that were inserted later might have skipped over that item, so searches for those items would terminate prematurely at the hole left by the deleted item. One solution to this problem is to rehash all the items for which this problem could arise—those between the deleted one and the next unoccupied position to the right. Figure 14.8 shows an example illustrating this process; Program 14.5 is an implementation. In a sparse table, this repair process will require only a few rehash operations, at most. Another way to implement deletion is to replace the deleted key with a sentinel key that can serve as a placeholder for searches but can be identified and reused for insertions (see Exercise 14.33).

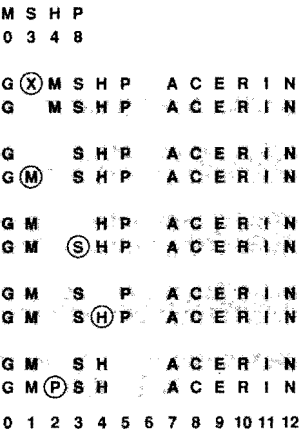


Figure 14.8
Deletion in a linear-probing hash table

This diagram shows the process of deleting the X from the table in Figure 14.7. The second line shows the result of just taking the X out of the table, and is an unacceptable final result because the M and the P are cut off from their hash positions by the empty table position left by the X. Thus, we reinsert the M, S, H, and P (the keys to the right of the X in the same cluster), in that order, using the hash values given at the top and resolving collisions with linear probing. The M fills the hole left by the X, then the S and the H hash into the table without collisions, then the P winds up in position 2.

Program 14.5 Deletion in a linear-probing hash table

To delete an item with a given key, we search for such an item and replace it with `NULLitem`. Then, we need to correct for the possibility that some item that lies to the right of the now-unoccupied position originally hashed to that position or to its left, because the vacancy would terminate a search for such an item. Therefore, we reinsert all the items in the same cluster as the deleted item and to that item's right. Since the table is less than half full, the number of items that are reinserted will be small, on the average.

```
void STdelete(Item item)
{ int j, i = hash(key(item), M); Item v;
  while (!null(i))
    if eq(key(item), key(st[i])) break;
    else i = (i+1) % M;
  if (null(i)) return;
  st[i] = NULLitem; N--;
  for (j = i+1; !null(j); j = (j+1) % M, N--)
    { v = st[j]; st[j] = NULLitem; STinsert(v); }
}
```

Exercises

- ▷ **14.24** How long could it take, in the worst case, to insert N keys into an initially empty table, using linear probing?
- ▷ **14.25** Give the contents of the hash table that results when you insert items with the keys `E A S Y Q U T I O N` in that order into an initially empty table of size $M = 16$ using linear probing. Use the hash function $11k \bmod M$ to transform the k th letter of the alphabet into a table index.
- 14.26** Do Exercise 14.25 for $M = 10$.
- **14.27** Write a program that inserts 10^5 random nonnegative integers less than 10^6 into a table of size 10^5 using linear probing, and that plots the total number of probes used for each 10^3 consecutive insertions.
- 14.28** Write a program that inserts $N/2$ random integers into a table of size N using linear probing, then computes the average cost of a search miss in the resulting table from the cluster lengths, for $N = 10^3, 10^4, 10^5$, and 10^6 .
- 14.29** Write a program that inserts $N/2$ random integers into a table of size N using linear probing, then computes the average cost of a search hit in the resulting table, for $N = 10^3, 10^4, 10^5$, and 10^6 . Do *not* search for all the keys at the end (keep track of the cost of constructing the table).

A	S	E	R	C	H	I	N	G	X	M	P	L
7	3	9	9	8	4	11	7	10	12	0	8	6
1	3	1	5	5	5	3	3	2	3	5	4	2

							(A)					
			(S)				A					
			S				A			(E)		
(R)			S				A			E		
R			S				A		(C)	E		
R			S		(H)		A		C	E		
R			S		H		A		C	E		(I)
R			S		H		A		C	E	(N)	I
R			S		H		A		C	E	N	I
R			S		H		A		C	E	N	I
R		(X)	S		H		A		C	E	N	I
(M)	R	X	S		H		A		C	E	N	I
M	R	X	S		H		(P)	A	C	E	N	I
M	R	X	S		H	(L)	P	A	C	E	N	I
0	1	2	3	4	5	6	7	8	9	10	11	12

Figure 14.9
Double hashing

This diagram shows the process of inserting the keys A S E R C H I N G X M P L into an initially empty hash table with open addressing, using the hash values given at the top and resolving collisions with double hashing. The first and second hash values for each key appear in the two rows below that key. As in Figure 14.7, table positions that are probed are unshaded. The A goes into position 7, then the S goes into position 3, then the E goes into position 9, as in Figure 14.7, but the R goes into position 1 after the collision at position 9, using its second hash value of 5 for the probe increment after collision. Similarly, P goes into position 6 on the final insertion after collisions at positions 8, 12, 3, 7, 11, and 2, using its second hash value 4 as the probe increment.

- **14.30** Run experiments to determine whether the average cost of search hits or search misses changes as a long sequence of alternating random insertions and deletions using Programs 14.4 and 14.5 is made in a hash table of size $2N$ with N keys, for $N = 10, 100$, and 1000 , and for up to N^2 insertion-deletion pairs for each N .

14.4 Double Hashing

The operative principle of linear probing (and indeed of any hashing method) is a guarantee that, when we are searching for a particular key, we look at every key that hashes to the same table address (in particular, the key itself, if it is in the table). In an open addressing scheme, however, other keys are typically also examined, particularly when the table begins to fill up. In the example depicted in Figure 14.7, a search for N involves looking at C, E, R, and I, none of which had the same hash value. What is worse, insertion of a key with one hash value can drastically increase the search times for keys with other hash values: in Figure 14.7, the insertion of M caused increased search times for positions 7–12 and 0–1. This phenomenon is called *clustering* because it has to do with the process of cluster formation. It can make linear probing run slowly for nearly full tables.

Fortunately, there is an easy way to virtually eliminate the clustering problem: *double hashing*. The basic strategy is the same as for linear probing; the only difference is that, instead of examining each successive table position following a collision, we use a second hash function to get a fixed increment to use for the probe sequence. An implementation is given in Program 14.6.

The second hash function must be chosen with some care, since otherwise the program may not work at all. First, we must exclude the case where the second hash function evaluates to 0, since that would lead to an infinite loop on the very first collision. Second, it is important that the value of the second hash function be relatively prime to the table size, since otherwise some of the probe sequences could be very short (for example, consider the case where the table size is twice the value of the second hash function). One way to enforce this policy is to make M prime and to choose a second hash function

Program 14.6 Double hashing

Double hashing is the same as linear probing except that we use a second hash function to determine the search increment to use after each collision. The search increment must be nonzero, and the table size and the search increment should be relatively prime. The `STdelete` function for linear probing (see Program 14.5) does *not* work with double hashing, because any key might be in many different probe sequences.

```
void STinsert(Item item)
{ Key v = key(item);
  int i = hash(v, M);
  int k = hashtwo(v, M);
  while (!null(i)) i = (i+k) % M;
  st[i] = item; N++;
}

Item STsearch(Key v)
{ int i = hash(v, M);
  int k = hashtwo(v, M);
  while (!null(i))
    if eq(v, key(st[i])) return st[i];
    else i = (i+k) % M;
  return NULLitem;
}
```

that returns values that are less than M . In practice, a simple second hash function such as

```
#define hashtwo(v) ((v % 97)+1)
```

will suffice for many hash functions, when the table size is not small. Also in practice, any loss in efficiency that is due to this simplification is not likely to be noticeable, much less to be significant. If the table is huge and sparse, the table size itself does not need to be prime because just a few probes will be used for every search (although we might want to test for and abort long searches to guard against an infinite loop, if we cut this corner (see Exercise 14.38)).

Figure 14.9 shows the process of building a small table with double hashing; Figure 14.10 shows that double hashing results in many

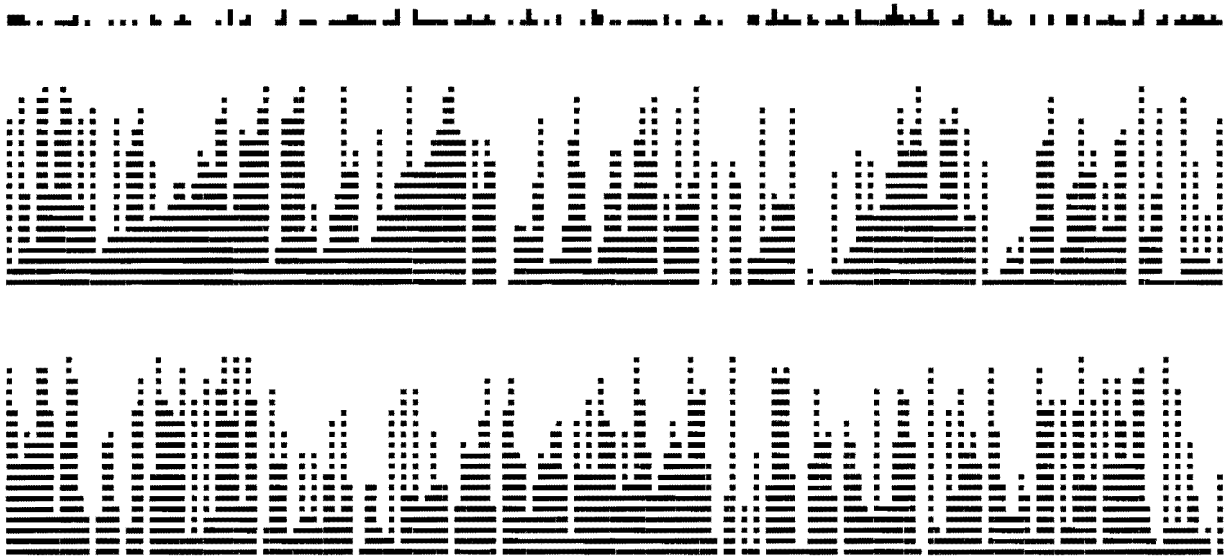


Figure 14.10
Clustering

These diagrams show the placement of records as we insert them into a hash table using linear probing (center) and double hashing (bottom), with the key value distribution shown at the top. Each line shows the result of inserting 10 records. As the table fills, the records cluster together into sequences separated by empty table positions. Long clusters are undesirable because the average cost of searching for one of the keys in the cluster is proportional to the cluster length. With linear probing, the longer clusters are, the more likely they are to increase in length, so a few long clusters dominate as the table fills up. With double hashing, this effect is much less pronounced, and the clusters remain relatively short.

fewer clusters (which are therefore much shorter) than the clusters left by linear probing.

Property 14.4 When collisions are resolved with double hashing, the average number of probes required to search in a hash table of size M that contains $N = \alpha M$ keys is

$$\frac{1}{\alpha} \ln\left(\frac{1}{1-\alpha}\right) \quad \text{and} \quad \frac{1}{1-\alpha}$$

for hits and misses, respectively.

These formulas are the result of a deep mathematical analysis done by Guibas and Szemerédi (see reference section). The proof is based on showing that double hashing is nearly equivalent to a more complicated *random hashing* algorithm where we use a key-dependent sequence of probe positions with each probe equally likely to hit each table position. This algorithm is only an approximation to double hashing for many reasons: for example, we take pains in double hashing to ensure that we try every table position once, but random hashing could examine the same table position more than once. Still, for sparse tables, the probabilities of collisions for the two methods are similar. We are interested in both: Double hashing is easy to implement, whereas random hashing is easy to analyze.

The average cost of a search miss for random hashing is given by the equation

$$1 + \frac{N}{M} + \left(\frac{N}{M}\right)^2 + \left(\frac{N}{M}\right)^3 + \dots = \frac{1}{1 - (N/M)} = \frac{1}{1 - \alpha}.$$

The expression on the left is the sum of the probability that a search miss uses more than k probes, for $k = 0, 1, 2, \dots$ (and is equal to the average from elementary probability theory). A search always uses one probe, then needs a second probe with probability N/M , a third probe with probability $(N/M)^2$, and so forth. We can also use this formula to compute the following approximation to the average cost of a search hit in a table with N keys:

$$\frac{1}{N} \left(1 + \frac{1}{1 - (1/M)} + \frac{1}{1 - (2/M)} + \dots + \frac{1}{1 - ((N-1)/M)} \right).$$

Each key in the table is equally likely to be hit; the cost of finding a key is the same as the cost of inserting it; and the cost of inserting the j th key in the table is the cost of a search miss in a table of $j-1$ keys, so this formula is the average of those costs. Now, we can simplify and evaluate this sum by multiplying the top and bottom of all the fractions by M :

$$\frac{1}{N} \left(1 + \frac{M}{M-1} + \frac{M}{M-2} + \dots + \frac{M}{M-N+1} \right)$$

and further simplify to get the result

$$\frac{M}{N} (H_M - H_{M-N}) \approx \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right),$$

since $H_M \approx \ln M$. ■

The precise nature of the relationship between the performance of double hashing and the random-hashing ideal that was proven by Guibas and Szemerédi is an asymptotic result that need not be relevant for practical table sizes; moreover, the results rest on the assumption that the hash functions return random values. Still, the asymptotic formulas in Property 14.5 are accurate predictors of the performance of double hashing in practice, even when we use an easy-to-compute second hash function such as $(v \% 97) + 1$. As do the corresponding formulas for linear probing, these formulas approach infinity as α approaches 1, but they do so much more slowly.



Figure 14.11
Costs of open-addressing search

These plots show the costs of building a hash table of size 1000 by inserting keys into an initially empty table using linear probing (top) and double hashing (bottom). Each bar represents the cost of 20 keys. The gray curves show the costs predicted by theoretical analysis (see Properties 14.4 and 14.5).

The contrast between linear probing and double hashing is illustrated clearly in Figure 14.11. Double hashing and linear probing have similar performance for sparse tables, but we can allow the table to become more nearly full with double hashing than we can with linear probing before performance degrades. The following table summarizes the expected number of probes for search hits and misses with double hashing:

load factor (α)	1/2	2/3	3/4	9/10
search hit	1.4	1.6	1.8	2.6
search miss	1.5	2.0	3.0	5.5

Search misses are always more expensive than hits, and both require only a few probes, on the average, even in a table that is nine-tenths full.

Looking at the same results in another way, double hashing allows us to use a smaller table than we would need with linear probing to get the same average search times.

Property 14.5 *We can ensure that the average cost of all searches is less than t probes by keeping the load factor less than $1 - 1/\sqrt{t}$ for linear probing and less than $1 - 1/t$ for double hashing.*

Set the equations for search misses in Property 14.4 and Property 14.5 equal to t , and solve for α . ■

For example, to ensure that the average number of probes for a search is less than 10, we need to keep the table at least 32 percent empty for linear probing, but only 10 percent empty for double hashing. If we have 10^5 items to process, we need space for just another 10^4 items to be able to do unsuccessful searches with fewer than 10 probes. By contrast, separate chaining would require more than 10^5 links, and BSTs would require twice that many.

The method of Program 14.5 for implementing the *delete* operation (rehash the keys that might have a search path containing the item to be deleted) breaks down for double hashing, because the deleted key might be in many different probe sequences, involving keys throughout the table. Thus, we have to resort to the other method that we considered at the end of Section 12.3: We replace the deleted item with a sentinel that marks the table position as occupied but does not match any key (see Exercise 14.33).

Like linear probing, double hashing is not an appropriate basis for implementing a full-function symbol table ADT where we need to support the *sort* or *select* operations.

Exercises

- ▷ **14.31** Give the contents of the hash table that results when you insert items with the keys `E A S Y Q U T I O N` in that order into an initially empty table of size $M = 16$ using double hashing. Use the hash function $11k \bmod M$ for the initial probe and the second hash function $(k \bmod 3) + 1$ for the search increment (when the key is the k th letter of the alphabet).
- ▷ **14.32** Answer Exercise **14.31** for $M = 10$
 - 14.33** Implement deletion for double hashing, using a sentinel item.
 - 14.34** Modify your solution to Exercise **14.27** to use double hashing.
 - 14.35** Modify your solution to Exercise **14.28** to use double hashing.
 - 14.36** Modify your solution to Exercise **14.29** to use double hashing.
- **14.37** Implement an algorithm that approximates random hashing, by providing the key as a seed to an in-line random number generator (as in Program **14.2**).
- 14.38** Suppose that a table of size 10^6 is half full, with occupied positions chosen at random. Estimate the probability that all positions with indices divisible by 100 are occupied.
- ▷ **14.39** Suppose that you have a bug in your double-hashing code such that one or both of the hash functions always return the same value (not 0). Describe what happens in each of these situations: (i) when the first one is wrong (ii) when the second one is wrong, and (iii) when both are wrong.

14.5 Dynamic Hash Tables

As the number of keys in a hash table increases, search performance degrades. With separate chaining, the search time increases gradually—when the number of keys in the table doubles, the search time doubles. The same is true of open-addressing methods such as linear probing and double hashing for sparse tables, but the cost increases dramatically as the table fills up, and, worse, we reach a point where no more keys can be inserted at all. This situation is in contrast to search trees, which accommodate growth naturally. For example, in a red-black tree, the search cost increases only slightly (by one comparison) whenever the number of nodes in the tree doubles.

Program 14.7 Dynamic hash insertion (for linear probing)

This implementation of `STinsert` for linear probing (see Program 14.4) handles an arbitrary number of keys by doubling the size of the table each time that the table becomes half full. Doubling requires that we allocate memory for the new table, rehash all the keys into the new table, then free the memory for the old table. The function `init` is an internal version of `STinit`; the ADT initialization `STinit` can be changed to start the table size `M` at 4 or any larger value. This same approach can be used for double hashing or separate chaining.

```
void expand();
void STinsert(Item item)
{ Key v = key(item);
  int i = hash(v, M);
  while (!null(i)) i = (i+1) % M;
  st[i] = item;
  if (N++ >= M/2) expand();
}
void expand()
{ int i; Item *t = st;
  init(M+M);
  for (i = 0; i < M/2; i++)
    if (key(t[i]) != key(NULLitem))
      STinsert(t[i]);
  free(t);
}
```

One way to accomplish growth in a hash table is to double the table's size when it begins to fill up. Doubling the table is an expensive operation because everything in the table has to be reinserted, but it is an operation that is performed infrequently. Program 14.7 is an implementation of growth by doubling for linear probing. An example is depicted in Figure 14.12. The same solution also works for double hashing, and the basic idea applies to separate chaining as well (see Exercise 14.46). Each time that the table gets more than half full, we expand the table by doubling it in size. After the first expansion, the table is always between one-quarter and one-half full, so the search cost is less than three probes, on the average. Furthermore, although the operation of rebuilding the table is expensive, it happens

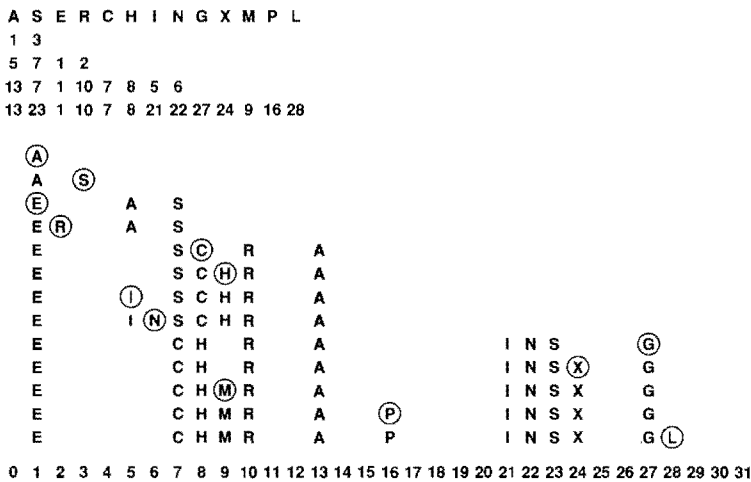


Figure 14.12
Dynamic hash-table expansion

This diagram shows the process of inserting the keys A S E R C H I N G X M P L into a dynamic hash table that expands by doubling, using the hash values given at the top and resolving collisions with linear probing. The four rows beneath the keys give the hash values when the table size is 4, 8, 16, and 32. The table size starts at 4, doubles to 8 for the E, to 16 for the C and to 32 for the G. All keys are rehashed and reinserted when the table size doubles. All insertions are into sparse tables (less than one-quarter full for reinsertion, between one-quarter and one-half full otherwise), so there are few collisions.

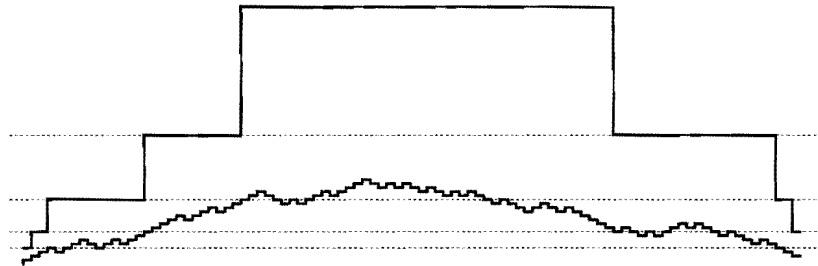
so infrequently that its cost represents only a constant fraction of the total cost of building the table.

Another way to express this concept is to say that the average cost *per insertion* is less than four probes. This assertion is not the same as saying that each insertion requires less than four probes on the average; indeed, we know that those insertions that cause the table to double will require a large number of probes. This argument is a simple example of *amortized analysis*: We cannot guarantee that each and every operation will be fast for this algorithm, but we can guarantee that the average cost per operation will be low.

Although the total cost is low, the performance profile for insertions is erratic: Most operations are extremely fast, but certain rare operations require about as much time as the whole previous cost of building the table. As a table grows from 1 thousand to 1 million keys, this slowdown will happen about 10 times. This kind of behavior is acceptable in many applications, but it might not be appropriate when absolute performance guarantees are desirable or required. For example, while a bank or an airline might be willing to suffer the consequences of keeping a customer waiting for so long on 10 out of every 1 million transactions, long waits might be catastrophic in other applications, such as an online system implementing a large financial transaction-processing system or in an air-traffic control system.

Figure 14.13
Dynamic hashing

This diagram shows the number of keys in the table (bottom) and the table size (top) when we insert keys into and delete them from a dynamic hash table using an algorithm that doubles the table when an insert makes it half full and halves the table when a deletion makes it one-eighth full. The table size is initialized at 4 and is always a power of 2 (dotted lines in the figure are at powers of 2). The table size changes when the curve tracing the number of keys in the table crosses a dotted line for the first time after having crossed a different dotted line. The table is always between one-eighth and one-half full.



If we support the *delete* ADT operation, then it may be worthwhile to contract the table by halving it as it shrinks (see Exercise 14.44), with one proviso: The thresholds for shrinking have to be separated from those for growing, because otherwise a small number of *insert* and *delete* operations could cause a sequence of doubling and halving operations even for huge tables.

Property 14.6 *A sequence of t search, insert, and delete symbol-table operations can be executed in time proportional to t and with memory usage always within a constant factor of the number of keys in the table.*

We use linear probing with growth by doubling whenever an *insert* causes the number of keys in the table to be half the table size, and we use shrinkage by halving whenever a *delete* causes the number of keys in the table to be one-eighth the table size. In both cases, after the table is rebuilt to size N , it has $N/4$ keys. Then, $N/4$ *insert* operations must be executed before the table doubles again (by reinsertion of $N/2$ keys into a table of size $2N$), and $N/8$ *delete* operations must be executed before the table halves again (by reinsertion of $N/8$ keys into a table of size $N/2$). In both cases, the number of keys reinserted is within a factor of 2 of the number of operations that we performed to bring the table to the point of being rebuilt, so the total cost is linear. Furthermore, the table is always between one-eighth and one-fourth full (see Figure 14.13), so the average number of probes for each operation is less than 3, by Property 14.4. ■

This method is appropriate for use in a symbol-table implementation for a general library where usage patterns are unpredictable, because it can handle tables of all sizes in a reasonable way. The primary drawback is the cost of rehashing and allocating memory

when the table expands and shrinks; in the typical case, when searches predominate, the guarantee that the table is sparse leads to excellent performance. In Chapter 16, we shall consider another approach that avoids rehashing and is suitable for huge external search tables.

Exercises

- ▷ **14.40** Give the contents of the hash table that results when you insert items with the keys `E A S Y Q U T I O N` in that order into an initially empty table of initial size $M = 4$ that is expanded with doubling whenever half full, with collisions resolved using linear probing. Use the hash function $11k \bmod M$ to transform the k th letter of the alphabet into a table index.
- 14.41** Would it be more economical to expand a hash table by tripling (rather than doubling) the table in size when the table is half full?
- 14.42** Would it be more economical to expand a hash table by tripling the table in size when the table is one-third full (rather than doubling the table in size when the table is half full)?
- 14.43** Would it be more economical to expand a hash table by doubling the table in size when the table is three-quarters (rather than half) full?
- 14.44** Add to Program 14.7 a *delete* function that deletes an item as in Program 14.4 but then contracts the table by halving it if the deletion leaves it seven-eighths empty.
- **14.45** Implement a version of Program 14.7 for separate chaining that increases the table size by a factor of 10 each time the average list length is equal to 10.
- 14.46** Modify Program 14.7 and your implementation from Exercise 14.44 to use double hashing with lazy deletion (see Exercise 14.33). Make sure that your program takes into account the number of dummy items, as well as the number of empty positions, in making the decisions whether to expand or contract the table.

14.6 Perspective

The choice of the hashing method that is best suited for a particular application depends on many different factors, as we have discussed when examining the methods. All the methods can reduce the symbol-table *search* and *insert* functions to constant-time operations, and all are useful for a broad variety of applications. Roughly, we can characterize the three major methods (linear probing, double hashing, and separate chaining) as follows: Linear probing is the fastest of the three

(if sufficient memory is available to ensure that the table is sparse), double hashing makes the most efficient use of memory (but requires extra time, to compute the second hash function), and separate chaining is the easiest to implement and deploy (provided that a good storage allocator is available). Table 14.1 gives empirical data and commentary on the performance of the algorithms.

The choice between linear probing and double hashing depends primarily on the cost of computing the hash function and on the load factor of the table. For sparse tables (small α), both methods use only a few probes, but double hashing could take more time because it has to compute two hash functions for long keys. As α approaches 1, double hashing far outperforms linear probing, as we saw in Figure 14.11.

Comparing linear probing and double hashing against separate chaining is more complicated, because we have to account precisely for memory usage. Separate chaining uses extra memory for links; the open-addressing methods use extra memory implicitly within the table to terminate probe sequences. The following concrete example illustrates the situation: Suppose that we have a table of M lists built with separate chaining, that the average length of the lists is 4, and that items and links each occupy a single machine word. The assumption that items and links take the same amount of space is justified in many situations because we would replace huge items with links to the items. With these assumptions, the table uses $9M$ words of memory ($4M$ for items and $5M$ for links), and delivers an average search time of 2 probes. But linear probing for $4M$ items in a table of size $9M$ requires just $(1 + 1/(1 - 4/9))/2 = 1.4$ probes for a search hit, a value that is 30 percent faster than separate chaining for the same amount of space; and linear probing for $4M$ items in a table of size $6M$ requires 2 probes for a search hit (on the average), and thus uses 33 percent less space than separate chaining for the same amount of time. Furthermore, we can use a dynamic method such as Program 14.7 to ensure that the table can grow while staying sparsely populated.

The argument in the previous paragraph indicates that it is not normally justifiable to choose separate chaining over open addressing on the basis of performance. However, separate chaining with a fixed M is often chosen in practice for a host of other reasons: it is easy to implement (particularly *delete*); it requires little extra memory for items that have preallocated link fields for use by symbol-table and other

ADTs that may need them; and, although its performance degrades as the number of items in the table grows, the degradation is graceful, and takes place in a manner that is unlikely to harm the application because it still is a factor of M faster than sequential search.

Many other hashing methods have been developed that have application in special situations. Although we cannot go into details, we consider three examples briefly to illustrate the nature of specially adapted hashing methods.

One class of methods moves items around during insertion in double hashing to make successful search more efficient. In fact, Brent developed a method for which the average time for a successful search can be bounded by a constant, even in a full table (*see reference section*). Such a method might be useful in applications where search hits are the predominant operation.

Another method, called *ordered hashing*, exploits ordering to reduce the cost for unsuccessful search in linear probing to be close to the cost for successful search. In standard linear probing, we stop the search when we find an empty table position or an item with a key equal to the search key; in ordered hashing, we stop the search when we find an item with a key greater than or equal to the search key (the table must be constructed cleverly if this procedure is to work) (*see reference section*). This improvement by introducing ordering in the table is on the same order as that we achieved by ordering the lists in separate chaining. This method is designed for applications where search misses predominate.

A symbol table that has a fast search miss and somewhat slower search hit can be used to implement an *exception dictionary*. For example, a text-processing system might have an algorithm for hyphenating words that works well for most words, but does not work for bizarre cases (such as “bizarre”). Only a few words in a huge document are likely to be in the exception dictionary, so nearly all the searches are likely to be misses.

These examples are only a few of a large number of algorithmic improvements that have been suggested for hashing. Many of these improvements are interesting and have important applications. Our usual cautions must be raised against premature use of advanced methods except when the requirements are serious and the performance/complexity tradeoffs are carefully considered, because separate

Table 14.1 Empirical study of hash-table implementations

These relative timings for building and searching symbol tables from random sequences of 32-bit integers confirm that hashing is significantly faster than tree search for keys that are easily hashed. Among the hashing methods, double hashing is slower than separate chaining and linear probing for sparse tables (because of the cost of computing the second hash function) but is much faster than linear probing as the table fills, and is the only one of the methods that can provide fast search using only a small amount of extra memory. Dynamic hash tables built with linear probing and expansion by doubling are more costly to construct than are other hash tables because of memory allocation and rehashing, but certainly lead to the fastest search times, and represent the method of choice when search predominates and when the number of keys cannot be predicted accurately in advance.

<i>N</i>	construction					search misses				
	R	H	P	D	P*	R	H	P	D	P*
1250	1	0	5	3	0	1	1	0	1	0
2500	3	1	3	4	2	1	1	0	0	0
5000	6	1	4	4	3	2	1	0	1	0
12500	14	6	5	5	5	6	1	2	2	1
25000	34	9	7	8	11	16	5	3	4	3
50000	74	18	11	12	22	36	15	8	8	8
100000	182	35	21	23	47	84	45	23	21	15
150000		54	40	36	138		99	89	52	21
160000		58	43	44	147		115	133	66	23
170000		68	55	45	136		121	226	85	25
180000		65	61	50	152		133	449	125	27
190000		79	106	59	155		144	2194	261	30
200000	407	84			159	186	156			33

Key:

- R Red-black BST (Programs 12.7 and 13.6)
- H Separate chaining (Program 14.3 with table size 20000)
- P Linear probing (Program 14.4 with table size 200000)
- D Double hashing (Program 14.6 with table size 200000)
- P Linear probing with expansion by doubling (Program 14.7)

chaining, linear probing and double hashing are simple, efficient, and acceptable for most applications.

The problem of implementing an exception dictionary is an example of an application where we can recast our algorithm slightly to optimize performance for the most frequently performed operation—in this case *search miss*. For example, suppose that we have a 1000-item exception dictionary, have 1 million items to look up in the dictionary, and expect virtually all the searches to end as misses. This situation might arise if the items were bizarre English-language words or random 32-bit integers. One way to proceed is to hash all the words to, say, 15-bit hash values (table size about 2^{16}). The 1000 exceptions occupy 1/64 of the table, and most of the 1 million searches end immediately with search misses, finding the empty table position on the first probe. But if the table contains 32-bit words, we can do much better by converting it into a bit-exception table and using 20-bit hash values. If we have a search miss (as we do most of the time), we finish the search with one bit test; a search hit requires a secondary test in a smaller table. The exceptions occupy 1/1000 of the table; search misses are by far the most likely operation; and we accomplish the task with 1 million directly indexed bit tests. This solution exploits the basic idea that a hash function produces a short *certificate* that represents a key—an essential concept that is useful in applications other than symbol-table implementations.

Hashing is preferred to the binary-tree structures of Chapters 12 and 13 as the symbol-table implementation for many applications, because it is somewhat simpler and can provide optimal (constant) search times, if the keys are of a standard type or are sufficiently simple that we can be confident of developing a good hash function for them. The advantages of binary-tree structures over hashing are that the trees are based on a simpler abstract interface (no hash function need be designed); the trees are dynamic (no advance information on the number of insertions is needed); the trees can provide guaranteed worst-case performance (everything could hash to the same place even in the best hashing method); and the trees support a wider range of operations (most important, *sort* and *select*). When these factors are not important, hashing is certainly the search method of choice, with one more important proviso: When keys are long strings, we can build

them into data structures that can provide for search methods even faster than hashing. Such structures are the subject of Chapter 15.

Exercises

- ▷ **14.47** For 1 million integer keys, compute the hash-table size that makes each of the three hashing methods (separate chaining, linear probing, and double hashing) use the same number of key comparisons as BSTs for a search miss, on the average, counting the hash-function computation as a comparison.
- ▷ **14.48** For 1 million integer keys, compute the number of comparisons for each of the three hashing methods (separate chaining, linear probing, and double hashing) for a search miss, on the average, when they can use a total of 3 million words of memory (as BSTs would).
- 14.49** Implement a symbol-table ADT with fast *search miss* as described in the text, using separate chaining for secondary testing.

Radix Search

SEVERAL SEARCH METHODS proceed by examining the search keys one small piece at a time, rather than using full comparisons between keys at each step. These methods, called *radix-search methods*, operate in a manner entirely analogous to the radix-sorting methods that we discussed in Chapter 10. They are useful when the pieces of the search keys are easily accessible, and they can provide efficient solutions to a variety of practical search tasks.

We use the same abstract model that we used in Chapter 10: Depending on the context, a *key* may be a *word* (a fixed-length sequence of bytes) or a *string* (a variable-length sequence of bytes). We treat keys that are words as numbers represented in a base- R number system, for various values of R (the *radix*), and work with individual digits of the numbers. We can view C strings as variable-length numbers terminated by a special symbol so that, for both fixed- and variable-length keys, we can base all our algorithms on the abstract operation “extract the i th digit from a key,” including a convention to handle the case that the key has fewer than i digits.

The principal advantages of radix-search methods are that the methods provide reasonable worst-case performance without the complication of balanced trees; they provide an easy way to handle variable-length keys; some of them allow space savings by storing part of the key within the search structure; and they can provide fast access to data, competitive with both binary search trees and hashing. The disadvantages are that some of the methods can make inefficient use of space, and that, as with radix sorting, performance can suffer if efficient access to the bytes of the keys is not available.

First, we examine several search methods that proceed by examining the search keys 1 bit at a time, using them to travel through binary tree structures. We examine a series of methods, each one correcting a problem inherent in the previous one, culminating in an ingenious method that is useful for a variety of search applications.

Next, we examine generalizations to R -way trees. Again, we examine a series of methods, culminating in a flexible and efficient method that can support a basic symbol-table implementation and numerous extensions.

In radix search, we usually examine the most significant digits of the keys first. Many of the methods directly correspond to MSD radix-sorting methods, in the same way that BST-based search corresponds to quicksort. In particular, we shall see the analog to the linear-time sorts of Chapter 10—constant-time search methods based on the same principle.

At the end of the chapter, we consider the specific application of using radix-search structures to build indexes for large text strings. The methods that we consider provide natural solutions for this application, and help to set the stage for us to consider more advanced string-processing tasks in Part 5.

```
A 00001
S 10011
E 00101
R 10010
C 00011
H 01000
I 01001
N 01110
G 00111
X 11000
M 01101
P 10000
L 01100
```

Figure 15.1
Binary representation of
single-character keys

As we did in Chapter 10, we use the 5-bit binary representation of i to represent the i th letter in the alphabet, as shown here for several sample keys, for the small examples in the figures in this chapter. We consider the bits as numbered from 0 to 4, from left to right.

15.1 Digital Search Trees

The simplest radix-search method is based on use of *digital search trees* (DSTs). The *search* and *insert* algorithms are identical to binary tree search except for one difference: We branch in the tree not according to the result of the comparison between the full keys, but rather according to selected bits of the key. At the first level, the leading bit is used; at the second level, the second leading bit is used; and so on, until an external node is encountered. Program 15.1 is an implementation of *search*; the implementation of *insert* is similar. Rather than using `less` to compare keys, we assume that the `digit` function is available to access individual bits in keys. This code is virtually the same as the code for binary tree search (see Program 12.7), but has substantially different performance characteristics, as we shall see.

We saw in Chapter 10 that we need to pay particular attention to equal keys in radix sorting; the same is true in radix search. Generally, we assume in this chapter that all the key values to appear in the

Program 15.1 Binary digital search tree

To develop a symbol-table implementation using DSTs, we modify the implementations of *search* and *insert* in the standard BST implementation (see Program 12.7) as shown in this implementation of *search*. Rather than doing a full key comparison, we decide whether to move left or right on the basis of testing a single bit (the leading bit) of the key. The recursive function calls have a third argument so that we can move the bit position to be tested to the right as we move down the tree. We use the *digit* operation to test bits, as discussed in Section 10.1. These same changes apply to implementation of *insert*; otherwise, we use all the code from Program 12.7.

```

Item searchR(link h, Key v, int w)
{
    Key t = key(h->item);
    if (h == z) return NULLitem;
    if (eq(v, t) return h->item;
    if (digit(v, w) == 0)
        return searchR(h->l, v, w+1);
    else return searchR(h->r, v, w+1);
}

Item STsearch(Key v)
{
    return searchR(head, v, 0);
}

```

symbol table are distinct. We can do so without loss of generality because we can use one of the methods discussed in Section 12.1 to support applications that have records with duplicate keys. It is important to focus on distinct key values in radix search, because key values are intrinsic components of several of the data structures that we shall consider.

Figure 15.1 gives binary representations for the one-letter keys used in other figures in the chapter. Figure 15.2 gives an example of insertion into a DST; Figure 15.3 shows the process of inserting keys into an initially empty tree.

The bits of the keys control search and insertion, but note that DSTs do not have the ordering property that characterizes BSTs. That is, it is *not* necessarily the case that nodes to the left of a given node have smaller keys or that nodes to the right have larger keys, as would be the case in a BST with distinct keys. It is true that keys on the left of a given node are smaller than keys on the right—if the node is at

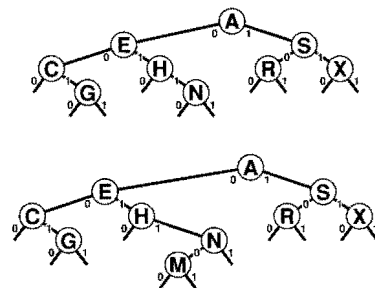


Figure 15.2
Digital search tree and insertion

In an unsuccessful search for $M = 01101$ in this sample digital search tree (top), we move left at the root (since the first bit in the binary representation of the key is 0) then right (since the second bit is 1), then right, then left, to finish at the null left link below N . To insert M (bottom), we replace the null link where the search ended with a link to the new node, just as we do with BST insertion.

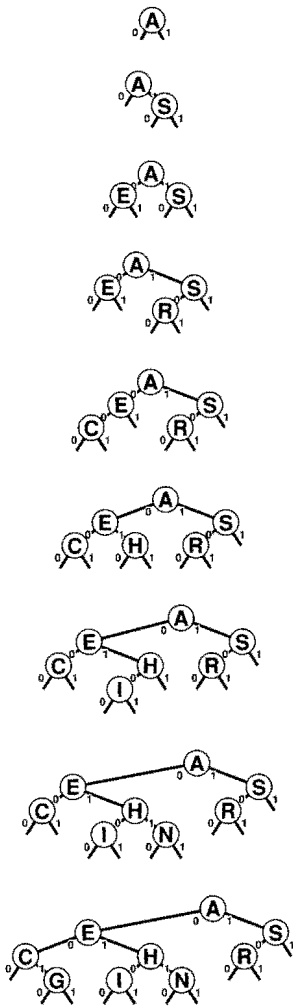


Figure 15.3
Digital search tree construction

This sequence depicts the result of inserting the keys A S E R C H I N G into an initially empty digital search tree.

level k , they all agree in the first k bits, but the next bit is 0 for the keys on the left and is 1 for the keys on the right—but the node's key could itself be the smallest, largest, or any value in between of all the keys in that node's subtree.

DSTs are characterized by the property that each key is *some-where* along the path specified by the bits of the key (in order from left to right). This property is sufficient for the *search* and *insert* implementations in Program 15.1 to operate properly.

Suppose that the keys are words of a fixed length, all consisting of w bits. Our requirement that keys are distinct implies that $N \leq 2^w$, and we normally assume that N is significantly smaller than 2^w , since otherwise key-indexed search (see Section 12.2) would be the appropriate algorithm to use. Many practical problems fall within this range. For example, DSTs are appropriate for a symbol table containing up to 10^5 records with 32-bit keys (but perhaps not as many as 10^6 records), or for any number of 64-bit keys. Digital tree search also works for variable-length keys; we defer considering that case in detail to Section 15.2, where we consider a number of other alternatives as well.

The worst case for trees built with digital search is much better than that for binary search trees, if the number of keys is large and the key lengths are small relative to the number of keys. The length of the longest path in a digital search tree is likely to be relatively small for many applications (for example, if the keys comprise random bits). In particular, the longest path is certainly limited by the length of the longest key; moreover, if the keys are of a fixed length, then the search time is limited by the length. Figure 15.4 illustrates this fact.

Property 15.1 *A search or insertion in a digital search tree requires about $\lg N$ comparisons on the average, and about $2 \lg N$ comparisons in the worst case, in a tree built from N random keys. The number of comparisons is never more than the number of bits in the search key.*

We can establish the stated average-case and worst-case results for random keys with an argument similar to one given for a more natural problem in the next section, so we leave this proof for an exercise there (see Exercise 15.29). It is based on the simple intuitive notion that the unseen portion of a random key should be equally likely to begin with a 0 bit as a 1 bit, so half should fall on either side of any node. Each

time that we move down the tree, we use up a key bit, so no search in a digital search tree can require more comparisons than there are bits in the search key. For the typical condition where we have w -bit words and the number of keys N is far smaller than the total possible number of keys 2^w , the path lengths are close to $\lg N$, so the number of comparisons is far smaller than the number of bits in the keys for random keys. ■

Figure 15.5 shows a large digital search tree made from random 7-bit keys. This tree is nearly perfectly balanced. DSTs are attractive in many practical applications because they provide near-optimal performance even for huge problems, with little implementation effort. For example, a digital search tree built from 32-bit keys (or four 8-bit characters) is guaranteed to require fewer than 32 comparisons, and a digital search tree built from 64-bit keys (or eight 8-bit characters) is guaranteed to require fewer than 64 comparisons, even if there are billions of keys. For large N , these guarantees are comparable to the guarantee provided by red–black trees, but are achieved with about the same implementation effort as is required for standard BSTs (which can promise only guaranteed performance proportional to N^2). This feature makes the use of digital search trees an attractive alternative to use of balanced trees in practice for implementing the *search* and *insert* symbol-table functions, *provided* that efficient access to key bits is available.

Exercises

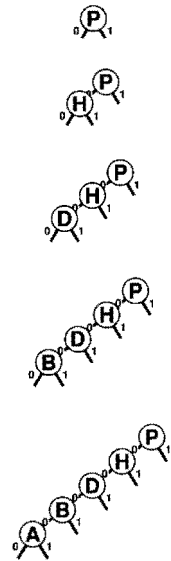
▷ 15.1 Draw the DST that results when you insert items with the keys EASY QUESTION in that order into an initially empty tree, using the binary encoding given in Figure 15.1.

15.2 Give an insertion sequence for the keys A B C D E F G that results in a perfectly balanced DST that is also a valid BST.

15.3 Give an insertion sequence for the keys A B C D E F G that results in a perfectly balanced DST with the property that every node has a key smaller than those of all the nodes in its subtree.

▷ 15.4 Draw the DST that results when you insert items with the keys 0101-0011 0000111 00100001 01010001 11101100 00100001 10010101 01001010 in that order into an initially empty tree.

15.5 Can we keep records with duplicate keys in DSTs, in the same way that we can in BSTs? Explain your answer.



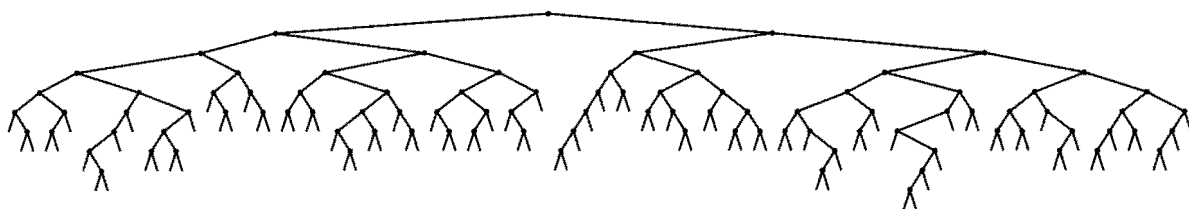


Figure 15.5
Digital search tree example

This digital search tree, built by insertion of about 200 random keys, is as well-balanced as its counterparts in Chapter 15.

- 15.6 Run empirical studies to compare the height and internal path length of a DST built by insertion of N random 32-bit keys into an initially empty tree with the same measures of a standard binary search tree and a red-black tree (Chapter 13) built from the same keys, for $N = 10^3, 10^4, 10^5$, and 10^6 .
- 15.7 Give a full characterization of the worst-case internal path length of a DST with N distinct w -bit keys.
- 15.8 Implement the *delete* operation for a DST-based symbol table.
- 15.9 Implement the *select* operation for a DST-based symbol table.
- 15.10 Describe how you could compute the height of a DST made from a given set of keys, in linear time, without building the DST.

15.2 Tries

In this section, we consider a search tree that allows us to use the bits of the keys to guide the search, in the same way that DSTs do, but that keeps the keys in the tree in order, so that we can support recursive implementations of *sort* and other symbol-table functions, as we did for BSTs. The idea is to store keys only at the bottom of the tree, in leaf nodes. The resulting data structure has a number of useful properties and serves as the basis for several effective search algorithms. It was first discovered by de la Briandais in 1959, and, because it is useful for retrieval, it was given the name *trie* by Fredkin in 1960. Ironically, in conversation, we usually pronounce this word “try-ee” or just “try,” so as to distinguish it from “tree.” For consistency with the nomenclature that we have been using, we perhaps should use the name “binary search trie,” but the term *trie* is universally used and understood. We consider the basic binary version in this section, an important variation in Section 15.3, and the basic multiway version and variations in Sections 15.4 and 15.5.

We can use tries for keys that are either a fixed number of bits or are variable-length bitstrings. To simplify the discussion, we start by

assuming that no search key is the prefix of another. For example, this condition is satisfied when the keys are of fixed length and are distinct.

In a trie, we keep the keys in the *leaves* of a binary tree. Recall from Section 5.4 that a leaf in a tree is a node with no children, as distinguished from an external node, which we interpret as a null child. In a binary tree, a leaf is an internal node whose left and right links are both null. Keeping keys in leaves instead of internal nodes allows us to use the bits of the keys to guide the search, as we did with DSTs in Section 15.1, while still maintaining the basic invariant at each node that all keys whose current bit is 0 fall in the left subtree and all keys whose current bit is 1 fall in the right subtree.

Definition 15.1 A trie is a binary tree that has keys associated with each of its leaves, defined recursively as follows: The trie for an empty set of keys is a null link; the trie for a single key is a leaf containing that key; and the trie for a set of keys of cardinality greater than one is an internal node with left link referring to the trie for the keys whose initial bit is 0 and right link referring to the trie for the keys whose initial bit is 1, with the leading bit considered to be removed for the purpose of constructing the subtrees.

Each key in the trie is stored in a leaf, on the path described by the leading bit pattern of the key. Conversely, each leaf contains the only key in the trie that begins with the bits defined by the path from the root to that leaf. Null links in nodes that are not leaves correspond to leading-bit patterns that do not appear in any key in the trie. Therefore, to search for a key in a trie, we just branch according to its bits, as we did with DSTs, but we do not do comparisons at internal nodes. We start at the left of the key and the top of the trie, and take the left link if the current bit is 0 and the right link if the current bit is 1, moving one bit position to the right in the key. A search that ends on a null link is a miss; a search that ends on a leaf can be completed with one key comparison, since that node contains the only key in the trie that could be equal to the search key. Program 15.2 is an implementation of this process.

To insert a key into a trie, we first perform a search, as usual. If the search ends on a null link, we replace that link with a link to a new leaf containing the key, as usual. But if the search ends on a leaf, we need to continue down the trie, adding an internal node for every bit

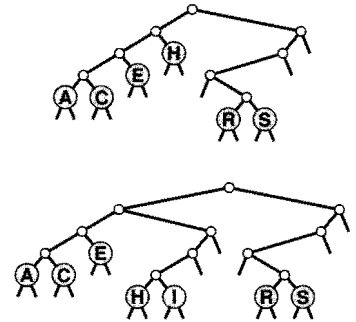


Figure 15.6
Trie search and insertion

Keys in a trie are stored in leaves (nodes with both links null); null links in nodes that are not leaves correspond to bit patterns not found in any keys in the trie.

In a successful search for the key $H = 01000$ in this sample trie (top), we move left at the root (since the first bit in the binary representation of the key is 0), then right (since the second bit is 1), where we find H , which is the only key in the trie that begins with 01. None of the keys in the trie begin with 101 or 11; these bit patterns lead to the two null links in the trie that are in non-leaf nodes.

To insert I (bottom), we need to add three non-leaf nodes: one corresponding to 01, with a null link corresponding to 011; one corresponding to 010, with a null link corresponding to 0101; and one corresponding to 0100 with $H = 01000$ in a leaf on its left and $I = 01001$ in a leaf on its right.

Program 15.2 Trie search

This function uses the bits of the key to control the branching on the way down the trie, in the same way as in Program 15.1 for DSTs. There are three possible outcomes: if the search reaches a leaf (with both links null), then that is the unique node in the trie that could contain the record with key v , so we test whether that node indeed contains v (search hit) or some key whose leading bits match v (search miss). If the search reaches a null link, then the parent's other link must not be null, so there is some other key in the trie that differs from the search key in the corresponding bit, and we have a search miss. This code assumes that the keys are distinct, and (if the keys may be of different lengths) that no key is a prefix of another. The item field is not used in non-leaf nodes.

```

Item searchR(link h, Key v, int w)
{
    if (h == z) return NULLitem;
    if ((h->l == z) && (h->r == z))
        return eq(v, key(h->item)) ? h->item : NULLitem;
    if (digit(v, w) == 0)
        return searchR(h->l, v, w+1);
    else return searchR(h->r, v, w+1);
}

Item STsearch(Key v)
{ return searchR(head, v, 0); }

```

where the search key and the key that was found agree, ending with both keys in leaves as children of the internal node corresponding to the first bit position where they differ. Figure 15.6 gives an example of trie search and insertion; Figure 15.7 shows the process of constructing a trie by inserting keys into an initially empty trie. Program 15.3 is a full implementation of the insertion algorithm.

We do not access null links in leaves, and we do not store items in non-leaf nodes, so we could save space in a C implementation by using union to define nodes as being one of these two types (see Exercise 15.20). For the moment, we will take the simpler route of using the single node type that we have been using for BSTs, DSTs, and other binary tree structures, with internal nodes characterized by null keys and leaves characterized by null links, knowing that we could

Program 15.3 Trie insertion

To insert a new node into a trie, we search as usual, then distinguish the two cases that can occur for a search miss. If the miss was not on a leaf, then we replace the null link that caused us to detect the miss with a link to a new node, as usual. If the miss was on a leaf, then we use a function `split` to make one new internal node for each bit position where the search key and the key found agree, finishing with one internal node for the leftmost bit position where the keys differ. The `switch` statement in `split` converts the 2 bits that it is testing into a number to handle the four possible cases. If the bits are the same (case $00_2 = 0$ or $11_2 = 3$), then we continue splitting; if the bits are different (case $01_2 = 1$ or $10_2 = 2$), then we stop splitting.

```

void STinit()
{ head = (z = NEW(NULLitem, 0, 0, 0)); }

link split(link p, link q, int w)
{ link t = NEW(NULLitem, z, z, 2);
  switch(digit(p->item, w)*2 + digit(q->item, w))
  {
    case 0: t->l = split(p, q, w+1); break;
    case 1: t->l = p; t->r = q; break;
    case 2: t->r = p; t->l = q; break;
    case 3: t->r = split(p, q, w+1); break;
  }
  return t;
}

link insertR(link h, Item item, int w)
{ Key v = key(item);
  if (h == z) return NEW(item, z, z, 1);
  if ((h->l == z) && (h->r == z))
    { return split(NEW(item, z, z, 1), h, w); }
  if (digit(v, w) == 0)
    h->l = insertR(h->l, item, w+1);
  else h->r = insertR(h->r, item, w+1);
  return h;
}

void STinsert(Item item)
{ head = insertR(head, item, 0); }

```

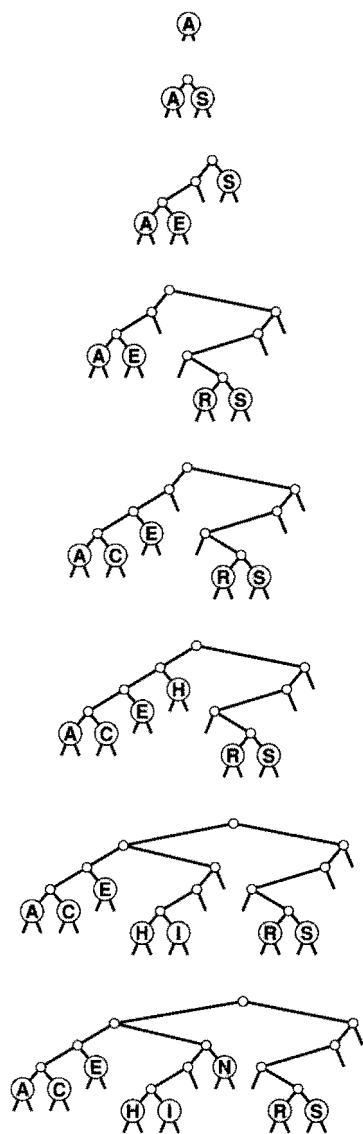


Figure 15.7
Trie construction

This sequence depicts the result of inserting the keys A S E R C H I N into an initially empty trie.

reclaim the space wasted because of this simplification, if desired. In Section 15.3, we will see an algorithmic improvement that avoids the need for multiple node types, and in Chapter 16, we will examine an implementation that uses union.

We now shall consider a number of basic properties of tries, which are evident from the definition and these examples.

Property 15.2 *The structure of a trie is independent of the key insertion order: There is a unique trie for any given set of distinct keys.*

This fundamental fact, which we can prove by induction on the subtrees, is a distinctive feature of tries: for all the other search tree structures that we have considered, the tree that we construct depends *both* on the set of keys *and* on the order in which we insert those keys.

■

The left subtree of a trie has all the keys that have 0 for the leading bit; the right subtree has all the keys that have 1 for the leading bit. This property of tries leads to an immediate correspondence with radix sorting: binary trie search partitions the file in exactly the same way as does binary quicksort (see Section 10.2). This correspondence is evident when we compare the trie in Figure 15.6 with Figure 10.4, the partitioning diagram for binary quicksort (after noting that the keys are slightly different); it is analogous to the correspondence between binary tree search and quicksort that we noted in Chapter 12.

In particular, unlike DSTs, tries *do* have the property that keys appear in order, so we can implement the *sort* and *select* symbol-table operations in a straightforward manner (see Exercises 15.17 and 15.18). Moreover, tries are as well-balanced as DSTs.

Property 15.3 *Insertion or search for a random key in a trie built from N random (distinct) bitstrings requires about $\lg N$ bit comparisons on the average. The worst-case number of bit comparisons is bounded only by the number of bits in the search key.*

We need to exercise care in analyzing tries because of our insistence that the keys be distinct, or, more generally, that no key be a prefix of another. One simple model that accommodates this assumption requires the keys to be a random (infinite) sequence of bits—we take the bits that we need to build the trie.

The average-case result then comes from the following probabilistic argument. The probability that each of the N keys in a random trie differ from a random search key in at least one of the leading t bits is

$$\left(1 - \frac{1}{2^t}\right)^N.$$

Subtracting this quantity from 1 gives the probability that one of the keys in the trie matches the search key in all of the leading t bits. In other words,

$$1 - \left(1 - \frac{1}{2^t}\right)^N$$

is the probability that the search requires more than t bit comparisons. From elementary probabilistic analysis, the sum for $t \geq 0$ of the probabilities that a random variable is $> t$ is the average value of that random variable, so the average search cost is given by

$$\sum_{t \geq 0} \left(1 - \left(1 - \frac{1}{2^t}\right)^N\right).$$

Using the elementary approximation $(1 - 1/x)^x \sim e^{-1}$, we find the search cost to be approximately

$$\sum_{t \geq 0} \left(1 - e^{-N/2^t}\right).$$

The summand is extremely close to 1 for approximately $\lg N$ terms with 2^t substantially smaller than N ; it is extremely close to 0 for all the terms with 2^t substantially greater than N ; and it is somewhere between 0 and 1 for the few terms with $2^t \approx N$. So the grand total is about $\lg N$. Computing a more precise estimate of this quantity requires using extremely sophisticated mathematics (see *reference section*). This analysis assumes that w is sufficiently large that we never run out of bits during a search, but taking into account the true value of w will only reduce the cost.

In the worst case, we could get two keys that have a huge number of equal bits, but this event happens with vanishingly small probability. The probability that the worst-case result quoted in Property 15.3 will not hold is exponentially small (see Exercise 15.28). ■

Another approach to analyzing tries is to generalize the approach that we used to analyze BSTs (see Property 12.6). The probability that

k keys start with a 0 bit and $N - k$ keys start with a 1 bit is $\binom{N}{k}/2^N$, so the external path length is described by the recurrence

$$C_N = N + \frac{1}{2^N} \sum_k \binom{N}{k} (C_k + C_{N-k}).$$

This recurrence is similar to the quicksort recurrence that we solved in Section 7.2, but it is much more difficult to solve. Remarkably, the solution is precisely N times the expression for the average search cost that we derived for Property 15.3 (see Exercise 15.25). Studying the recurrence itself gives insight into why tries have better balance than do BSTs: The probability is much higher that the split will be near the middle than that it will be anywhere else, so the recurrence is more like the mergesort recurrence (approximate solution $N \lg N$) than like the quicksort recurrence (approximate solution $2N \ln N$).

An annoying feature of tries, and another one that distinguishes them from the other types of search trees that we have seen, is the one-way branching required when keys have bits in common. For example, keys that differ in only the final bit always require a path whose length is equal to the key length, no matter how many keys there are in the tree, as illustrated in Figure 15.8. The number of internal nodes can be somewhat larger than the number of keys.

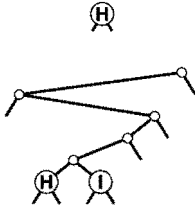


Figure 15.8
Binary trie worst case

This sequence depicts the result of inserting the keys $H = 01000$ and $I = 01001$ into an initially empty binary trie. As it is in DSTs (see Figure 15.4), the path length is limited by the length of the binary representation of the keys; as illustrated by this example, however, paths could be that long even with only two keys in the trie.

Property 15.4 *A trie built from N random w -bit keys has about $N/\ln 2 \approx 1.44N$ nodes on the average.*

By modifying the argument for Property 15.3, we can write the expression

$$\sum_{t \geq 0} \left(2^t \left(1 - \left(1 - \frac{1}{2^t} \right)^N \right) - N \left(1 - \frac{1}{2^t} \right)^{N-1} \right)$$

for the average number of nodes in an N -key trie (see Exercise 15.26). The mathematical analysis that yields the stated approximate value for this sum is much more difficult than the argument that we gave for Property 15.3, because many terms contribute values that are not 0 or 1 to the value of the sum (see reference section). ■

We can verify these results empirically. For example, Figure 15.9 shows a big trie, which has 44 percent more nodes than does the BST or the DST built with the same set of keys but nevertheless is well balanced, with a near-optimal search cost. Our first thought might be that the extra nodes would raise the average search cost substantially,

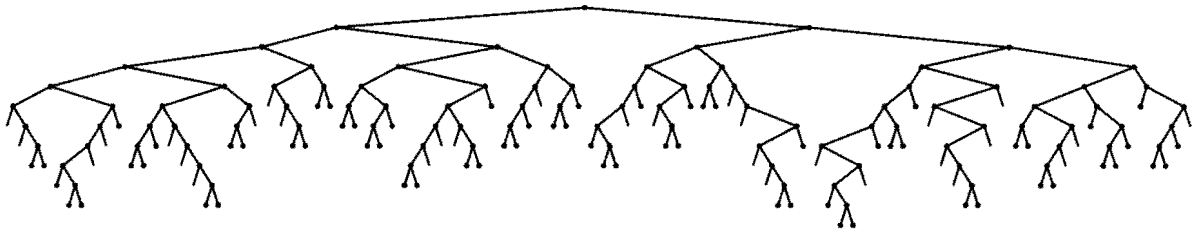


Figure 15.9
Trie example

This trie, built by inserting about 200 random keys, is well-balanced, but has 44 percent more nodes than might otherwise be necessary, because of one-way branching. (Null links on leaves are not shown.)

but this suspicion is not valid—for example, we would increase the average search cost by only 1 even if we were to double the number of nodes in a balanced trie.

For convenience in the implementations in Programs 15.2 and 15.3, we assumed that the keys are of fixed length and are distinct, so that we could be certain that the keys would eventually distinguish themselves and that the programs could process 1 bit at a time and never run out of key bits. For convenience in the analyses in Properties 15.2 and 15.3, we implicitly assumed that the keys have an arbitrary number of bits, so that they eventually distinguish themselves except with tiny (exponentially decaying) probability. A direct offshoot of these assumptions is that both the programs and the analyses apply when the keys are variable-length bitstrings, with a few caveats.

To use the programs as they stand for variable-length keys, we need to extend our restriction that the keys be distinct to say that no key be a prefix of another. This restriction is met automatically in some applications, as we shall see in Section 15.5. Alternatively, we could handle such keys by keeping information in internal nodes, because each prefix that might need to be handled corresponds to some internal node in the trie (see Exercise 15.30).

For sufficiently long keys comprising random bits, the average-case results of Properties 15.2 and 15.3 still hold. In the worst case, the height of a trie is still limited by the number of bits in the longest keys. This cost could be excessive if the keys are huge and perhaps have some uniformity, as might arise in encoded character data. In the next two sections, we consider methods of reducing trie costs for long keys. One way to shorten paths in tries is to collapse one-way branches into single links—we discuss an elegant and efficient way to accomplish this task in Section 15.3. Another way to shorten paths in

tries is to allow more than two links per node—this approach is the subject of Section 15.4.

Exercises

- ▷ 15.11 Draw the trie that results when you insert items with the keys `E A S Y Q U T I O N` in that order into an initially empty trie.
- 15.12 What happens when you use Program 15.3 to insert a record whose key is equal to some key already in the trie?
- 15.13 Draw the trie that results when you insert items with the keys `0101-0011 00000111 00100001 01010001 11101100 00100001 10010101 01001010` into an initially empty trie.
- 15.14 Run empirical studies to compare the height, number of nodes, and internal path length of a trie built by insertion of N random 32-bit keys into an initially empty trie with the same measures of a standard binary search tree and a red-black tree (Chapter 13) built from the same keys, for $N = 10^3$, 10^4 , 10^5 , and 10^6 (see Exercise 15.6).
- 15.15 Give a full characterization of the worst-case internal path length of a trie with N distinct w -bit keys.
- 15.16 Implement the *delete* operation for a trie-based symbol table.
- 15.17 Implement the *select* operation for a trie-based symbol table.
- 15.18 Implement the *sort* operation for a trie-based symbol table.
- ▷ 15.19 Write a program that prints out all keys in a trie that have the same initial t bits as a given search key.
- 15.20 Use the C `union` construct to develop implementations of *search* and *insert* using tries with non-leaf nodes that contain links but no items and with leaves that contain items but no links.
- 15.21 Modify Programs 15.3 and 15.2 to keep the search key in a machine register and to shift one bit position to access the next bit when moving down a level in the trie.
- 15.22 Modify Programs 15.3 and 15.2 to maintain a table of 2^r tries, for a fixed constant r , and to use the first r bits of the key to index into the table and the standard algorithms with the remainder of the key on the trie accessed. This change saves about r steps unless the table has a significant number of null entries.
- 15.23 What value should we choose for r in Exercise 15.22, if we have N random keys (which are sufficiently long that we can assume them to be distinct)?
- 15.24 Write a program to compute the number of nodes in the trie corresponding to a given set of distinct fixed-length keys, by sorting them and comparing adjacent keys in the sorted list.

- **15.25** Prove by induction that $N \sum_{t \geq 0} (1 - (1 - 2^{-t})^N)$ is the solution to the quicksort-like recurrence that is given after Property 15.3 for the external path length in a random trie.
- **15.26** Derive the expression given in Property 15.4 for the average number of nodes in a random trie.
- **15.27** Write a program to compute the average number of nodes in a random trie of N nodes and print the exact value, accurate to 10^{-3} , for $N = 10^3, 10^4, 10^5$, and 10^6 .
- **15.28** Prove that the height of a trie built from N random bitstrings is about $2 \lg N$. *Hint:* Consider the birthday problem (see Property 14.2).
- **15.29** Prove that the average cost of a search in a DST built from random keys is asymptotically $\lg N$ (see Properties 15.1 and 15.2).

15.30 Modify Programs 15.2 and 15.3 to handle variable-length bitstrings under the sole restriction that records with duplicate keys are not kept in the data structure. Assume that `bit(v, w)` yields the value `NULLdigit` if `w` is greater than the length of `v`.

15.31 Use a trie to build a data structure that can support an existence table ADT for w -bit integers. Your program should support the *initialize*, *insert*, and *search* operations, where *search* and *insert* take integer arguments, and *search* returns `NULLkey` for search miss and the argument it was given for search hit.

15.3 Patricia Tries

Trie-based search as described in Section 15.2 has two inconvenient flaws. First, the one-way branching leads to the creation of extra nodes in the trie, which seem unnecessary. Second, there are two different types of nodes in the trie, and that complicates the code somewhat. In 1968, Morrison discovered a way to avoid both of these problems, in a method that he named *patricia* (“practical algorithm to retrieve information coded in alphanumeric”). Morrison developed his algorithm in the context of string-indexing applications of the type that we shall consider in Section 15.5, but it is equally effective as a symbol-table implementation. Like DSTs, patricia tries allow search for N keys in a tree with just N nodes; like tries, they require only about $\lg N$ bit comparisons and one full key comparison per search, and they support other ADT operations. Moreover, these performance characteristics are independent of key length, and the data structure is suitable for variable-length keys.

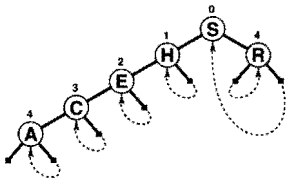


Figure 15.10
Patricia search

In a successful search for $R = 10010$ in this sample patricia trie (top), we move right (since bit 0 is 1), then left (since bit 4 is 0), which brings us to R (the only key in the tree that begins with $1^{***}0$). On the way down the tree, we check only the key bits indicated in the numbers over the nodes (and ignore the keys in the nodes). When we first reach a link that points up the tree, we compare the search key against the key in the node pointed to by the up link, since that is the only key in the tree that could be equal to the search key.

In an unsuccessful search for $I = 01001$, we move left at the root (since bit 0 of the key is 0), then take the right (up) link (since bit 1 is 1) and find that H (the only key in the trie that begins with 01) is not equal to I .

Starting with the standard trie data structure, we avoid one-way branching via a simple device: we put into each node the index of the bit to be tested to decide which path to take out of that node. Thus, we jump directly to the bit where a significant decision is to be made, bypassing the bit comparisons at nodes where all the keys in the subtree have the same bit value. Moreover, we avoid external nodes via another simple device: we store data in internal nodes and replace links to external nodes with links that point back upwards to the correct internal node in the trie. These two changes allow us to represent tries with binary trees comprising nodes with a key and two links (and an additional field for the index), which we call *patricia tries*. With patricia tries, we store keys in nodes as with DSTs, and we traverse the tree according to the bits of the search key, but we do not use the keys in the nodes on the way down the tree to control the search; we merely store them there for possible later reference, when the bottom of the tree is reached.

As hinted in the previous paragraph, it is easier to follow the mechanics of the algorithm if we first take note that we can regard standard tries and patricia tries as different representations of the same abstract trie structure. For example, the tries in Figure 15.10 and at the top in Figure 15.11, which illustrate search and insertion for patricia tries, represent the same abstract structure as do the tries in Figure 15.6. The search and insertion algorithms for patricia tries use, build, and maintain a concrete representation of the abstract trie data structure different from the search and insertion algorithms discussed in Section 15.2, but the underlying trie abstraction is the same.

Program 15.4 is an implementation of the patricia-trie search algorithm. The method differs from trie search in three ways: there are no explicit null links, we test the indicated bit in the key instead of the next bit, and we end with a search key comparison at the point where we follow a link up the tree. It is easy to test whether a link points up, because the bit indices in the nodes (by definition) increase as we travel down the tree. To search, we start at the root and proceed down the tree, using the bit index in each node to tell us which bit to examine in the search key—we go right if that bit is 1, left if it is 0. The keys in the nodes are not examined at all on the way down the tree. Eventually, an upward link is encountered: each upward link points to the unique key in the tree that has the bits that would cause

Program 15.4 Patricia-trie search

The recursive function `searchR` returns the unique node that could contain the record with key `v`. It travels down the trie, using the bits of the tree to control the search, but tests only 1 bit per node encountered—the one indicated in the `bit` field. It terminates the search when it encounters an external link, one which points up the tree. The search function `STsearch` calls `searchR`, then tests the key in that node to determine whether the search is a hit or a miss.

```
Item searchR(link h, Key v, int w)
{
    if (h->bit <= w) return h->item;
    if (digit(v, h->bit) == 0)
        return searchR(h->l, v, h->bit);
    else return searchR(h->r, v, h->bit);
}

Item STsearch(Key v)
{ Item t = searchR(head->l, v, -1);
  return eq(v, key(t)) ? t : NULLitem;
}
```

a search to take that link. Thus, if the key at the node pointed to by the first upward link encountered is equal to the search key, then the search is successful; otherwise, it is unsuccessful.

Figure 15.10 illustrates search in a patricia trie. For a miss due to the search taking a null link in a trie, the corresponding patricia trie search will take a course somewhat different from that of standard trie search, because the bits that correspond to one-way branching are not tested at all on the way down the trie. For a search ending at a leaf in a trie, the patricia-trie search ends up comparing against the same key as the trie search, but without examining the bits corresponding to one-way branching in the trie.

The implementation of insertion for patricia tries mirrors the two cases that arise in insertion for tries, as illustrated in Figure 15.11. As usual, we gain information on where a new key belongs from a search miss. For tries, the miss can occur either because of a null link or because of a key mismatch at a leaf. For patricia tries, we need to do more work to decide which type of insertion is needed, because

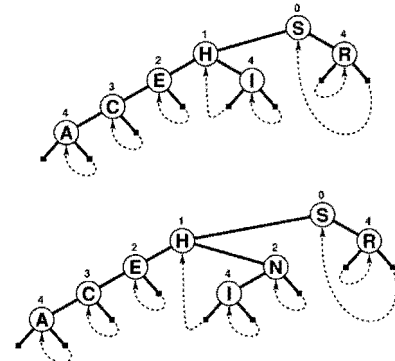


Figure 15.11
Patricia-trie insertion

To insert `I` into the sample patricia trie in Figure 15.10, we add a new node to check bit 4, since `H = 01000` and `I = 01001` differ in only that bit (top). On a subsequent search in the trie that comes to the new node, we want to check `H` (left link) if bit 4 of the search key is 0; if the bit is 1 (right link), the key to check is `I`.

To insert `N = 01110` (bottom), we add a new node in between `H` and `I` to check bit 2, since that bit distinguishes `N` from `H` and `I`.

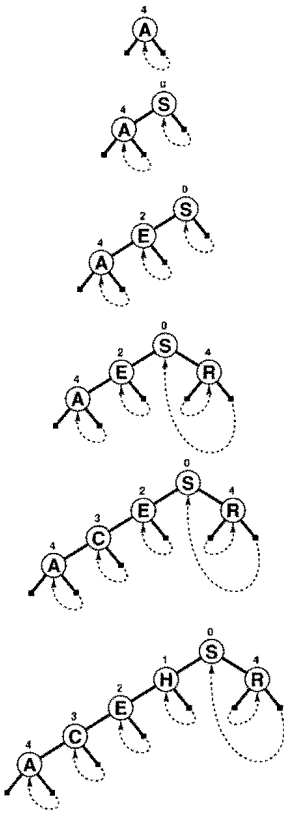


Figure 15.12
Patricia-trie construction

This sequence depicts the result of inserting the keys A S E R C H into an initially empty patricia trie. Figure 15.11 depicts the result of inserting I and then N into the tree at the bottom.

we skipped the bits corresponding to one-way branching during the search. A patricia-trie search always ends with a key comparison, and this key carries the information that we need. We find the leftmost bit position where the search key and the key that terminated the search differ, then search through the tree again, comparing that bit position against the bit positions in the nodes on the search path. If we come to a node that specifies a bit position higher than the bit position that distinguishes the key sought and the key found, then we know that we skipped a bit in the patricia-trie search that would have led to a null link in the corresponding trie search, so we add a new node for testing that bit. If we never come to a node that specifies a bit position higher than the one that distinguishes the key sought and the key found, then the patricia-trie search corresponds to a trie search ending in a leaf, and we add a new node that distinguishes the search key from the key that terminated the search. We always add just one node, which references the leftmost bit that distinguishes the keys, where standard trie insertion might add multiple nodes with one-way branching before reaching that bit. That new node, besides providing the bit-discrimination that we need, will also be the node that we use to store the new item. Figure 15.12 shows the initial stages of the construction of a sample trie.

Program 15.5 is an implementation of the patricia-trie-insertion algorithm. The code follows directly from the description in the previous paragraph, with the additional observation that we view links to nodes with bit indices that are not larger than the current bit index as links to external nodes. The insertion code merely tests this property of the links, but does not have to move keys or links around at all. The upward links in patricia tries seem mysterious at first, but the decisions about which links to use when each node is inserted are surprisingly straightforward. The end result is that using one node type rather than two simplifies the code substantially.

By construction, all external nodes below a node with bit index k begin with the same k bits (otherwise, we would have created a node with bit index less than k to distinguish two of them). Therefore, we can convert a patricia trie to a standard trie by creating the appropriate internal nodes between nodes where bits are skipped and by replacing links that point up the tree with links to external nodes (see Exercise 15.47). However, Property 15.2 does not quite hold for patricia

Program 15.5 Patricia-trie insertion

To insert a key into a patricia trie, we begin with a search. The function `searchR` from Program 15.4 gets us to a unique key in the tree that must be distinguished from the key to be inserted. We determine the leftmost bit position at which this key and the search key differ, then use the recursive function `insertR` to travel down the tree and to insert a new node containing `v` at that point.

In `insertR`, there are two cases, corresponding to the two cases illustrated in Figure 15.11. The new node could replace an internal link (if the search key differs from the key found in a bit position that was skipped), or an external link (if the bit that distinguishes the search key from the found key was not needed to distinguish the found key from all the other keys in the trie).

```
void STinit()
{ head = NEW(NULLitem, 0, 0, -1);
  head->l = head; head->r = head; }

link insertR(link h, Item item, int w, link p)
{ link x; Key v = key(item);
  if ((h->bit >= w) || (h->bit <= p->bit))
  {
    x = NEW(item, 0, 0, w);
    x->l = digit(v, x->bit) ? h : x;
    x->r = digit(v, x->bit) ? x : h;
    return x;
  }
  if (digit(v, h->bit) == 0)
    h->l = insertR(h->l, item, w, h);
  else h->r = insertR(h->r, item, w, h);
  return h;
}

void STinsert(Item item)
{ int i;
  Key v = key(item);
  Key t = key(searchR(head->l, v, -1));
  if (v == t) return;
  for (i = 0; digit(v, i) == digit(t, i); i++) ;
  head->l = insertR(head->l, item, i, head);
}
```

Program 15.6 Patricia-trie sort

This recursive procedure visits the records in a patricia trie in order of their keys. We imagine the items to be in (virtual) external nodes, which we can identify by testing when the bit index on the current node is not larger than the bit index on its parent. Otherwise, this program is a standard inorder traversal.

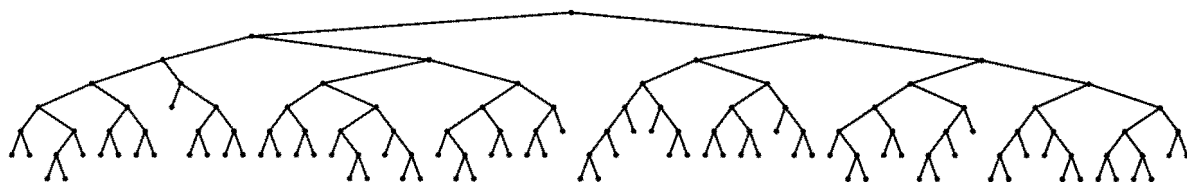
```
void sortR(link h, void (*visit)(Item), int w)
{
    if (h->bit <= w) { visit(h->item); return; }
    sortR(h->l, visit, h->bit);
    sortR(h->r, visit, h->bit);
}
void STsort(void (*visit)(Item))
{ sortR(head->l, visit, -1); }
```

tries, because the assignment of keys to internal nodes does depend on the order in which the keys are inserted. The structure of the internal nodes is independent of the key-insertion order, but external links and the placement of the key values are not.

An important consequence of the fact that a patricia trie represents an underlying standard trie structure is that we can use a recursive inorder traversal to visit the nodes in order, as demonstrated in the implementation given in Program 15.6. We visit just the external nodes, which we identify by testing for nonincreasing bit indices.

Patricia is the quintessential radix search method: it manages to identify the bits that distinguish the search keys and to build them into a data structure (with no surplus nodes) that quickly leads from any search key to the only key in the data structure that could be equal to the search key. Figure 15.13 shows the patricia trie for the same keys used to build the trie of Figure 15.9—the patricia trie not only has 44 percent fewer nodes than the standard trie, but also is nearly perfectly balanced.

Property 15.5 *Insertion or search for a random key in a patricia trie built from N random bitstrings requires about $\lg N$ bit comparisons on the average, and about $2 \lg N$ bit comparisons in the worst case. The number of bit comparisons is never more than the length of the key.*



This fact is an immediate consequence of Property 15.3, since paths in patricia tries are no longer than paths in the corresponding trie. The precise average-case analysis of patricia is difficult; it turns out that patricia involves one fewer comparison, on the average, than does a standard trie (*see reference section*). ■

Table 15.1 gives empirical data supporting the conclusion that DSTs, standard binary tries, and patricia tries have comparable performance (and that they provide search times comparable to or shorter than the balanced-tree methods of Chapter 13) when keys are integers, and certainly should be considered for symbol-table implementations even with keys that can be represented as short bitstrings, taking into account the various straightforward tradeoffs that we have noted.

Note that the search cost given in Property 15.5 does *not* grow with the key length. By contrast, the search cost in a standard trie typically does depend on the length of the keys—the first bit position that differs in two given keys could be arbitrarily far into the key. All the comparison-based search methods that we have considered also depend on the key length—if two keys differ in only their rightmost bit, then comparing them requires time proportional to their length. Furthermore, hashing methods *always* require time proportional to the key length for a search, to compute the hash function. But patricia immediately takes us to the bits that matter, and typically involves testing less than $\lg N$ of them. This effect makes patricia (or trie search with one-way branching removed) the search method of choice when the search keys are long.

For example, suppose that we have a computer that can efficiently access 8-bit bytes of data, and we have to search among millions of 1000-bit keys. Then patricia would require accessing only about 20 bytes of the search key for the search, plus one 125-byte equality comparison, whereas hashing would require accessing all 125 bytes of the search key to compute the hash function, plus a few equality com-

Figure 15.13
Patricia-trie example

This patricia trie, built by insertion of about 200 random keys, is equivalent to the trie of Figure 15.9 with one-way branching removed. The resulting tree is nearly perfectly balanced.

Table 15.1 Empirical study of trie implementations

These relative timings for construction and search in symbol tables with random sequences of 32-bit integers confirm that digital methods are competitive with balanced-tree methods, even for keys that are random bits. Performance differences are more remarkable when keys are long and are not necessarily random (see Table 15.2), or when careful attention is paid to making the key-bit-access code efficient (see Exercise 15.21).

<i>N</i>	construction				search hits			
	B	D	T	P	B	D	T	P
1250	1	1	1	1	0	1	1	0
2500	2	2	4	3	1	1	2	1
5000	4	5	7	7	3	2	3	2
12500	18	15	20	18	8	7	9	7
25000	40	36	44	41	20	17	20	17
50000	81	80	99	90	43	41	47	36
100000	176	167	269	242	103	85	101	92
200000	411	360	544	448	228	179	211	182

Key:

- B Red-black BST (Programs 12.7 and 13.6)
- D DST (Program 15.1)
- T Trie (Programs 15.2 and 15.3)
- P Patricia trie (Programs 15.4 and 15.5)

parisons, and comparison-based methods would require 20 to 30 full key comparisons. It is true that key comparisons, particularly in the early stages of a search, require only a few byte comparisons, but later stages typically involve many more bytes. We shall consider comparative performance of various methods for searching with lengthy keys again in Section 15.5.

Indeed, there needs to be no limit at all on the length of search keys for patricia. Patricia is particularly effective in applications with variable-length keys that are potentially huge, as we shall see in Section 15.5. With patricia, we generally can expect that the number of

bit inspections required for a search among N records, even with huge keys, will be roughly proportional to $\lg N$.

Exercises

15.32 What happens when you use Program 15.5 to insert a record whose key is equal to some key already in the trie?

▷ **15.33** Draw the patricia trie that results when you insert the keys EASYQU
TION in that order into an initially empty trie.

▷ **15.34** Draw the patricia trie that results when you insert the keys 01010011
00000111 00100001 01010001 11101100 00100001 10010101 01001010 in
that order into an initially empty trie.

○ **15.35** Draw the patricia trie that results when you insert the keys 01001010
10010101 00100001 11101100 01010001 00100001 00000111 01010011 in
that order into an initially empty trie.

15.36 Run empirical studies to compare the height and internal path length of a patricia trie built by insertion of N random 32-bit keys into an initially empty trie with the same measures of a standard binary search tree and a red-black tree (Chapter 13) built from the same keys, for $N = 10^3, 10^4, 10^5$, and 10^6 (see Exercises 15.6 and 15.14).

15.37 Give a full characterization of the worst-case internal path length of a patricia trie with N distinct w -bit keys.

▷ **15.38** Implement the *select* operation for a patricia-based symbol table.

• **15.39** Implement the *delete* operation for a patricia-based symbol table.

• **15.40** Implement the *join* operation for patricia-based symbol tables.

○ **15.41** Write a program that prints out all keys in a patricia trie that have the same initial t bits as a given search key.

15.42 Modify standard trie search and insertion (Programs 15.2 and 15.3) to eliminate one-way branching in the same manner as for patricia tries. If you have done Exercise 15.20, start with that program instead.

15.43 Modify patricia search and insertion (Programs 15.4 and 15.5) to maintain a table of 2^t tries, as described in Exercise 15.22.

15.44 Show that each key in a patricia trie is on its own search path, and is therefore encountered on the way down the tree during a *search* operation as well as at the end.

15.45 Modify patricia search (Program 15.4) to compare keys on the way down the tree to improve search-hit performance. Run empirical studies to evaluate the effectiveness of this change (see Exercise 15.44).

15.46 Use a patricia trie to build a data structure that can support an existence table ADT for w -bit integers (see Exercise 15.31).

• **15.47** Write programs that convert a patricia trie to a standard trie on the same keys, and vice versa.

15.4 Multiway Tries and TSTs

For radix sorting, we found that we could get a significant improvement in speed by considering more than 1 bit at a time. The same is true for radix search: By examining r bits at a time, we can speed up the search by a factor of r . However, there is a catch that makes it necessary for us to be more careful in applying this idea than we had to be for radix sorting. The problem is that considering r bits at a time corresponds to using tree nodes with $R = 2^r$ links, and that can lead to a considerable amount of wasted space for unused links.

In the (binary) tries of Section 15.2, the nodes corresponding to key bits have two links: one for the case when the key bit is 0, and the other for the case when the key bit is 1. The appropriate generalization is to R -ary tries, where we have nodes with R links corresponding to key digits, one for each possible digit value. Keys are stored in leaves (nodes with all links null). To search in an R -way trie, we start at the root and at the leftmost key digit, and use the key digits to guide us down the tree. We go down the i th link (and move to the next digit) if the digit value is i . If we reach a leaf, it contains the only key in the trie with leading digits corresponding to the path that we have traversed, so we can compare that key with the search key to determine whether we have a search hit or a search miss. If we reach a null link, we know that we have a search miss, because that link corresponds to a leading-digit pattern not found in any keys in the trie. Figure 15.14 shows a 10-way trie that represents a sample set of decimal numbers. As we discussed in Chapter 10, numbers typically seen in practice are distinguished with relatively few trie nodes. This same effect for more general types of keys is the basis for a number of efficient search algorithms.

Before doing a full symbol-table implementation with multiple node types and so forth, we begin our study of multiway tries by concentrating on the *existence-table* problem, where we have only keys (no records or associated information) and want to develop algorithms to *insert* a key into a data structure and to *search* the data structure to tell us whether or not a given key has been inserted. To use the same interface that we have been using for more general symbol-table implementations, we assume that `Key` is the same as `Item` and adopt the convention that the search function returns `NULLitem` for a search

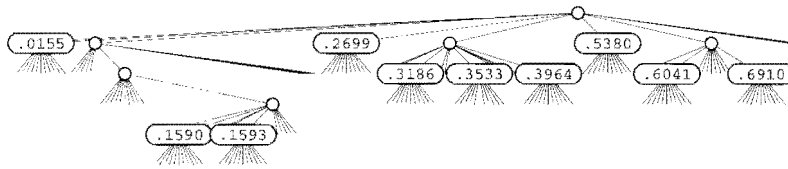


Figure 15.14
R-way trie for base-10 numbers

This figure depicts the trie that distinguishes the set of numbers

```
.396465048
.353336658
.318693642
.015583409
.159369371
.691004885
.899854354
.159072306
.604144269
.269971047
.538069659
```

miss and the search key for a search hit. This convention simplifies the code and clearly exposes the structure of the multiway tries that we shall be considering. In Section 15.5, we shall discuss more general symbol-table implementations, including string indexing.

Definition 15.2 *The existence trie corresponding to a set of keys is defined recursively as follows: The trie for an empty set of keys is a null link; and the trie for a nonempty set of keys is an internal node with links referring to the trie for each possible key digit, with the leading digit considered to be removed for the purpose of constructing the subtrees.*

For simplicity, we assume in this definition that no key is the prefix of another. Typically, we enforce this restriction by ensuring that the keys are distinct and either are of fixed length or have a termination character. The point of this definition is that we can use existence tries to implement existence tables, without storing *any* information within the trie; the information is all implicitly defined within the trie *structure*. Each node has $R + 1$ links (one for each possible character value plus one for the terminal character `NULLdigit`), and no other information. To search, we use the digits in the key to guide us down the trie. If we reach the link to `NULLdigit` at the same time that we run out of key digits, we have a search hit; otherwise we have a search miss. To insert a new key, we search until we reach a null link, then add nodes for each of the remaining characters in the key. Figure 15.15 is an example of a 27-way trie; Program 15.7 is an implementation of the basic (multiway) existence-trie search and insert procedures.

If the keys are of fixed length and are distinct, we can dispense with the link to the terminal character and can terminate searches when we reach the key length (see Exercise 15.54). We have already seen an example of this type of trie when we used tries to describe MSD sorting for fixed-length keys (Figure 10.10).

In one sense, this pure abstract representation of the trie structure is optimal, because it can support the *search* operation in time

(see Figure 12.1). Each node has 10 links (one for each possible digit). At the root, link 0 points to the trie for keys with first digit 0 (there is only one); link 1 points to the trie for keys with first digit 1 (there are two), and so forth. None of these numbers has first digit 4, 7, 8, or 9, so those links are null. There is only one number for each of the first digits 0, 2, and 5, so there is a leaf containing the appropriate number for each of those digits. The rest of the structure is built recursively, moving one digit to the right.

Program 15.7 Existence-trie search and insertion

This implementation of *search* and *insert* for multiway tries assumes that *Key* is identical to *Item* (and that *digit* is defined, as discussed in Section 10.1). It stores the keys implicitly within the structure of the trie. Each node contains *R* pointers to the next level down the trie. We follow the *i*th link at level *t* when the *t*th digit of the key is *i*. The *search* function returns the key given as the argument if it is in the table, and returns *NULLitem* otherwise.

```
typedef struct STnode *link;
struct STnode { link next[R]; };
static link head;
void STinit() { head = NULL; }
link NEW()
{ int i;
  link x = malloc(sizeof *x);
  for (i = 0; i < R; i++) x->next[i] = NULL;
  return x;
}
Item searchR(link h, Key v, int w)
{ int i = digit(v, w);
  if (h == NULL) return NULLitem;
  if (i == NULLdigit) return v;
  return searchR(h->next[i], v, w+1);
}
Item STsearch(Key v)
{ return searchR(head, v, 0); }
link insertR(link h, Item item, int w)
{ Key v = key(item);
  int i = digit(v, w);
  if (h == NULL) h = NEW();
  if (i == NULLdigit) return h;
  h->next[i] = insertR(h->next[i], v, w+1);
  return h;
}
void STinsert(Item item)
{ head = insertR(head, item, 0); }
```

proportional to the length of a key and in space proportional to the total number of characters in the key in the worst case. But the total amount of space used could be as high as nearly R links for each character, so we seek improved implementations. As we saw with binary tries, it is worthwhile to consider the pure trie structure as a particular representation of an underlying abstract structure that is a well-defined representation of our set of keys, and then to consider other representations of the same abstract structure that might lead to better performance.

Definition 15.3 A multiway trie is a multiway tree that has keys associated with each of its leaves, defined recursively as follows: The trie for an empty set of keys is a null link; the trie for a single key is a leaf containing that key; and the trie for a set of keys of cardinality greater than one is an internal node with links referring to tries for keys with each possible digit value, with the leading digit considered to be removed for the purpose of constructing the subtrees.

We assume that keys in the data structure are distinct and that no key is the prefix of another. To search in a standard multiway trie, we use the digits of the key to guide the search down the trie, with three possible outcomes. If we reach a null link, we have a search miss; if we reach a leaf containing the search key, we have a search hit; and if we reach a leaf containing a different key, we have a search miss. All leaves have R null links, so different representations for leaf nodes and non-leaf nodes are appropriate, as mentioned in Section 15.2. We consider such an implementation in Chapter 16, and we shall consider another approach to an implementation in this chapter. In either case, the analytic results from Section 15.3 generalize to tell us about the performance characteristics of standard multiway tries.

Property 15.6 Search or insertion in a standard R -ary trie built from N random bytestrings requires about $\log_R N$ byte comparisons, on the average. The number of links in a such a trie is about $RN/\ln R$. The number of byte comparisons for search or insertion in such a trie is no more than the number of bytes in the search key.

These results generalize those in Properties 15.3 and 15.4. We can establish them by substituting R for 2 in the proofs of those properties. As we mentioned, however, extremely sophisticated mathematics is involved in the precise analysis of these quantities. ■

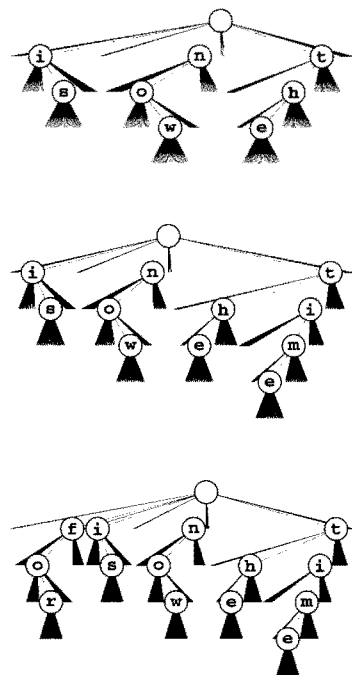


Figure 15.15
R-way existence trie search and insertion

The 26-way trie for the words *now*, *is*, and *the* (top) has nine nodes: the root plus one for each letter. The nodes are labeled in these diagrams, but we do not use explicit node labels in the data structure, because each node label can be inferred from the position of the link to it in its parents' link array.

To insert the key *time*, we branch off the existing node for *t* and add new nodes for *i*, *m*, and *e* (center); to insert the key *for*, we branch off the root and add new nodes for *f*, *o*, and *r*.

The performance characteristics listed in Property 15.6 represent an extreme example of a time–space tradeoff. On the one hand, there are a large number of unused null links—only a few nodes near the top use more than a few of their links. On the other hand, the height of a tree is small. For example, suppose that we take the typical value $R = 256$ and that we have N random 64-bit keys. Property 15.6 tells us that a search will take $(\lg N)/8$ character comparisons (8 at most) and that we will use fewer than $47N$ links. If plenty of space is available, this method provides an extremely efficient alternative. We could cut the search cost to 4 character comparisons for this example by taking $R = 65536$, but that would require over 5900 links.

We shall return to standard multiway tries in Section 15.5; in the remainder of this section, we shall consider an alternative representation of the tries built by Program 15.7: the *ternary search trie* (TST), which is illustrated in its full form in Figure 15.16. In a TST, each node has a character and *three* links, corresponding to keys whose current digits are less than, equal to, or greater than the node's character. Using this arrangement is equivalent to implementing trie nodes as binary search trees that use as keys the characters corresponding to non-null links. In the standard existence tries of Program 15.7, trie nodes are represented by $R + 1$ links, and we infer the character represented by each non-null link by its index. In the corresponding existence TST, all the characters corresponding to non-null links appear explicitly in nodes—we find characters corresponding to keys only when we are traversing the middle links.

The search algorithm for existence TSTs is so straightforward as nearly to write itself; the insertion algorithm is slightly more complicated, but mirrors directly insertion in existence tries. To search, we compare the first character in the key with the character at the root. If it is less, we take the left link; if it is greater, we take the right link; and if it is equal, we take the middle link and move to the next key character. In each case, we apply the algorithm recursively. We terminate with a search miss if we encounter a null link or if we encounter the end of the search key before encountering `NULLdigit` in the tree, and we terminate with a search hit if we traverse the middle link in a node whose character is `NULLdigit`. To insert a new key, we search, then add new nodes for the characters in the tail of the key, just as we did for tries. Program 15.8 gives the details of the implementation of

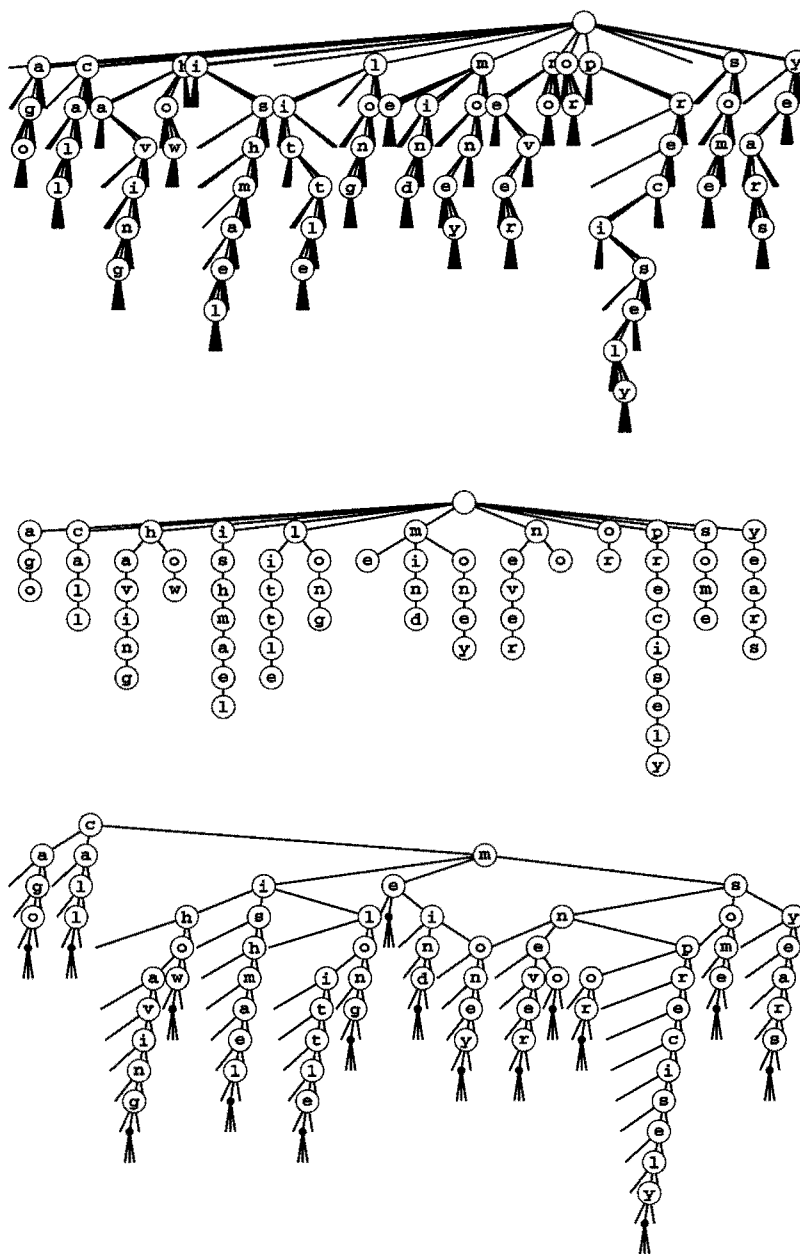


Figure 15.16
Existence-trie structures

These figures show three different representations of the existence trie for the 16 words *call me ish mael some years ago never mind how long precisely having little or no money*: The 26-way existence trie (top); the abstract trie with null links removed (center); and the TST representation (bottom). The 26-way trie has too many links, but the TST is an efficient representation of the abstract trie.

The top two tries assume that no key is the prefix of another. For example, adding the key *not* would result in the key *no* being lost. We can add a null character to the end of each key to correct this problem, as illustrated in the TST at the bottom.

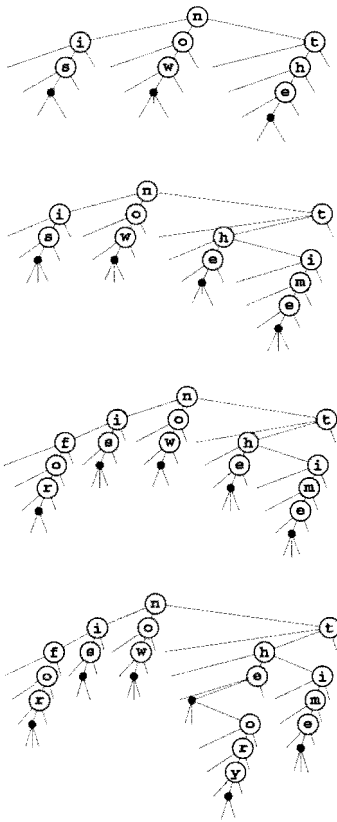


Figure 15.17
Existence TSTs

An existence TST has one node for each letter, but only 3 children per node, rather than 26. The top three trees in this figure are the RSTs corresponding to the insertion example in Figure 15.15, with the additional change that an end-of-key character is appended to each key. We can then remove the restriction that no key may be a prefix of another, so, for example, we can insert the key **theory** (bottom).

these algorithms, and Figure 15.17 has TSTs that correspond to the tries in Figure 15.15.

Continuing the correspondence that we have been following between search trees and sorting algorithms, we see that TSTs correspond to three-way radix sorting in the same way that BSTs correspond to quicksort, tries correspond to binary quicksort, and M -way tries correspond to M -way radix sorting. Figure 10.13, which describes the recursive call structure for three-way radix sort, is a TST for that set of keys. The null-links problem for tries corresponds to the empty-bins problem for radix sorting; three-way branching provides an effective solution to both problems.

We can make TSTs more efficient in their use of space by putting keys in leaves at the point where they are distinguished and by eliminating one-way branching between internal nodes as we did for patricia. At the end of this section, we examine an implementation based on the former change.

Property 15.7 *A search or insertion in a full TST requires time proportional to the key length. The number of links in a TST is at most three times the number of characters in all the keys.*

In the worst case, each key character corresponds to a full R -ary node that is unbalanced, stretched out like a singly linked list. This worst case is extremely unlikely to occur in a random tree. More typically, we might expect to do $\ln R$ or fewer byte comparisons at the first level (since the root node behaves like a BST on the R different byte values) and perhaps at a few other levels (if there are keys with a common prefix and up to R different byte values on the character following the prefix), and to do only a few byte comparisons for most characters (since most trie nodes are sparsely populated with non-null links). Search misses are likely to involve only a few byte comparisons, ending at a null link high in the trie, and search hits involve only about one byte comparison per search key character, since most of them are in nodes with one-way branching at the bottom of the trie.

Actual space usage is generally less than the upper bound of three links per character, because keys share nodes at high levels in the tree. We refrain from a precise average-case analysis because TSTs are most useful in practical situations where keys neither are random nor are derived from bizarre worst-case constructions. ■

Program 15.8 Existence-TST search and insertion

This code implements the same abstract trie algorithms as Program 15.7, but each node contains one digit and three links: one each for keys whose next digit is less than, equal to, or greater than the corresponding digit in the search key, respectively.

```

typedef struct STnode* link;
struct STnode { int d; link l, m, r; };
static link head;
void STinit() { head = NULL; }
link NEW(int d)
{ link x = malloc(sizeof *x);
  x->d = d; x->l = NULL; x->m = NULL; x->r = NULL;
  return x;
}
Item searchR(link h, Key v, int w)
{ int i = digit(v, w);
  if (h == NULL) return NULLitem;
  if (i == NULLdigit) return v;
  if (i < h->d) return searchR(h->l, v, w);
  if (i == h->d) return searchR(h->m, v, w+1);
  if (i > h->d) return searchR(h->r, v, w);
}
Item STsearch( Key v)
{ return searchR(head, v, 0); }
link insertR(link h, Item item, int w)
{ Key v = key(item);
  int i = digit(v, w);
  if (h == NULL) h = NEW(i);
  if (i == NULLdigit) return h;
  if (i < h->d) h->l = insertR(h->l, v, w);
  if (i == h->d) h->m = insertR(h->m, v, w+1);
  if (i > h->d) h->r = insertR(h->r, v, w);
  return h;
}
void STinsert(Key key)
{ head = insertR(head, key, 0); }

```

The prime virtue of using TSTs is that they adapt gracefully to irregularities in search keys that are likely to appear in practical applications. There are two main effects. First, keys in practical applications come from large character sets, and usage of particular characters in the character sets is far from uniform—for example, a particular set of strings is likely to use only a small fraction of the possible characters. With TSTs, we can use a 128- or 256-character encoding without having to worry about the excessive costs of nodes with 128- or 256-way branching, and without having to determine which sets of characters are relevant. Character sets for non-Roman alphabets can have thousands of characters—TSTs are especially appropriate for string keys that consist of such characters. Second, keys in practical applications often have a structured format, differing from application to application, perhaps using only letters in one part of the key, only digits in another part of the key, and special characters as delimiters (see Exercise 15.71). For example, Figure 15.18 gives a list of library call numbers from an online library database. For such keys, some of the trie nodes might be represented as unary nodes in the TST (for places where all keys have delimiters); some might be represented as 10-node BSTs (for places where all keys have digits); and still others might be represented as 26-node BSTs (for places where all keys have letters). This structure develops automatically, without any need for special analysis of the keys.

A second practical advantage of TST-based search over many other algorithms is that search misses are likely to be extremely efficient, even when the keys are long. Often, the algorithm uses just a few byte comparisons (and chases a few pointers) to complete a search miss. As we discussed in Section 15.3, a search miss in a hash table with N keys requires time proportional to the key length (to compute the hash function), and at least $\lg N$ key comparisons in a search tree. Even patricia requires $\lg N$ bit comparisons for a random search miss.

Table 15.2 gives empirical data in support of the observations in the previous two paragraphs.

A third reason that TSTs are attractive is that they support operations more general than the symbol-table operations that we have been considering. For example, Program 15.9 gives a program that allows particular characters in the search key to be unspecified, and prints all keys in the data structure that match the specified digits of

```
LDS___361_H_4
LDS___485_N_4_H_317
LDS___625_D_73_1986
LJN___679_N_48_1985
LQP___425_M_56_1991
LTK___6015_P_63_1988
LVM___455_M_67_1974
WAFR____5054____33
WKG____6875
WLSOC____2542____30
WPHIL____4060____2____55
WPHYS____39____1____30
WROM____5350____65____5
WUS____10706____7____10
WUS____12692____4____27
```

Figure 15.18
Sample string keys (library call numbers)

These keys from an online library database illustrate the variability of the structure found in string keys in applications. Some of the characters may appropriately be modeled as random letters, some may be modeled as random digits, and still others have fixed value or structure.

Table 15.2 Empirical study of search with string keys

These relative timings for construction and search in symbol tables with string keys such as the library call numbers in Figure 15.18 confirm that TSTs, although slightly more expensive to construct, provide the fastest search for search misses with string keys, primarily because the search does not require examination of all the characters in the key.

N	construction				search misses			
	B	H	T	T*	B	H	T	T*
1250	4	4	5	5	2	2	2	1
2500	8	7	10	9	5	5	3	2
5000	19	16	21	20	10	8	6	4
12500	48	48	54	97	29	27	15	14
25000	118	99	188	156	67	59	36	30
50000	230	191	333	255	137	113	70	65

Key:

- B Standard BST (Program 12.7)
- H Hashing with separate chaining ($M = N/5$) (Program 14.3)
- T TST (Program 15.8)
- T* TST with R^2 -way branch at root (Programs 15.10 and 15.11)

the search key. An example is depicted in Figure 15.19. Obviously, with a slight modification, we can adapt this program to visit all the matching keys in the way that we do for *sort*, rather than just to print them (see Exercise 15.57).

Several other similar tasks are easy to handle with TSTs. For example, we can visit all keys in the data structure that differ from the search key in at most one digit position (see Exercise 15.58). Operations of this type are expensive or impossible with other symbol-table implementations. We shall consider in detail these and many other problems where we do not insist on exact matches in a string search, in Part 5.

Patricia offers several of the same advantages; the main practical advantage of TSTs over patricia tries is that the former access key bytes rather than key bits. One reason that this difference represents

